

The Book of the Class for csc 453

Spring 2026

Phillip L. Nico
Department of Computer Science
California Polytechnic State University
San Luis Obispo, CA 93407
pnico@calpoly.edu

DRAFT(April 27, 2026 at 14:04)

Abstract

These are my lecture notes and nothing more. That means that they are full of omissions and errors and were meant only as a means of helping me remember what I intended to talk about in class. They may or may not bear any resemblance to what was actually said. They may, however, be useful in providing some small insight into the author's state of mind and approach to the course material.

Contents

0 Syllabus	1
1 Lecture: Introduction and Background	11
1.1 Robust Programming	12
1.2 Background: The Ballad of the Unknown Stuntman	12
1.3 Syllabus	12
1.4 How to succeed in this course	16
1.5 The Last Page	16
1.6 Foundations	16
1.6.1 What an operating system is	16
1.6.2 Virtualization and Transparency	17
1.6.3 What an operating system is not	17
1.7 Major subjects for the quarter	17
1.8 This quarter	18
1.9 About the lab	18
1.10 About the assignment	18
2 Lecture: History and Definition of an O.S.	20
2.1 Announcements	20
2.2 About the lab	21
2.3 About the assignment	21
2.4 Everything you wanted to know about C development	22
2.5 Aside: Review of 357	22
2.5.1 Review of Unix IO	22
2.5.2 Processes, etc.	22
2.5.3 Compilation	22
3 Lecture: History	23
3.1 Announcements	23
3.2 OS History	24
3.2.1 Ancient times	24
3.2.2 The middle ages	24
3.2.3 The renaissance: families	25
3.2.4 Modern Times	25
3.3 Defining of an Operating System: The System Calls	26
3.4 System Calls Again	26
3.5 System Call Mechanisms	26
3.5.1 How to do it	26
3.6 OS Pre-history: The boot process	27
3.6.1 How it all begins (on an Intel PC with a floppy)	27
3.6.2 How it continues	27
3.6.3 And onwards: How does the OS get control back?	27
3.6.4 Tool of the week: Make(1)	31

4	Lecture: The Process Model	32
4.1	Announcements	32
4.2	From last time	32
4.3	The Process (Users' View)	33
4.4	The Operating System's view: The context	33
4.5	Example of a context switch	34
4.6	System Calls Again	34
4.7	System Call Mechanisms	34
4.7.1	How to do it	34
4.7.2	And onwards	36
4.8	Onwards	36
4.9	The Process Model: a little deeper	36
4.10	Pseudoparallelism and nondeterminism	36
4.11	Possible process states	37
5	Lecture: Operating System Structures	38
5.1	Announcements	38
5.2	Onwards	38
5.3	The Process Model: a little deeper	39
5.4	Pseudoparallelism and nondeterminism	39
5.5	Possible process states	39
5.6	Scheduling	40
5.7	What about IPC?	40
5.8	Operating System Structures	41
5.8.1	Monolithic Systems	41
5.8.2	Layered Systems	42
5.8.3	Virtual Machines	42
5.8.4	Client-server model	42
5.9	The Layered Architecture of Minix	43
5.10	Example: Description a MINIX disk interrupt	43
6	Lecture: Intro. to Concurrency	45
6.1	Announcements	45
6.2	Once again: Processes	46
6.3	Lightweight Processes: Threads	46
6.4	Aside: Reentrancy	47
6.5	Introduction to Asgn2	47
6.5.1	Nine little functions	47
6.5.2	Demonstration: threading in action	48
6.5.3	Review: context switch	48
6.5.4	A thread's context: stack and registers	48
6.6	How to do it	52
7	Lecture: Concurrency and Synchronization	54
7.1	Announcements	54
7.2	Problems with parallelism: Race Conditions	54
7.2.1	Race condition, defined	55
7.3	Critical Sections	55

7.4	Mutual Exclusion	55
7.5	Busy Waiting: Software Only	55
7.6	Peterson's Solution	56
8	Lecture: Concurrency and Synchronization, cont.	58
8.1	Announcements	58
8.2	LWP	58
8.3	From last time: Review of Busy Waiting (Software only)	59
8.4	Onwards: Busy Waiting: With Hardware Support	59
8.5	Reflection	59
8.5.1	sleep() and wakeup()	60
8.6	Synchronization without busy waiting	61
8.7	Semaphores	61
8.7.1	Monitors: (Hoare 1974, Brinch Hansen 1975)	61
8.8	More Interprocess Communication	62
9	Lecture: More Synchronization	64
9.1	Announcements	64
9.2	LWP?	64
9.3	More Interprocess Communication	65
9.3.1	Further generalization: message passing	65
9.4	Classic IPC Problems	66
10	Lecture: Scheduling	68
10.1	Announcements	68
10.2	We talked a lot about LWP	68
10.3	So what: Scheduling	69
10.3.1	Process States	69
10.3.2	Policy vs. Mechanism	69
10.3.3	Process types	69
10.3.4	When to schedule	69
10.3.5	Evaluation Criteria	70
10.3.6	Non-preemptive scheduling: run-to-completion	70
10.3.7	Preemptive scheduling	71
11	Lecture: More Scheduling	72
11.1	Announcements	72
11.1.1	Preemptive scheduling	72
11.2	Minix architecture revisited	73
11.3	Minix 2 Scheduling	73
11.4	Minix 3 Scheduling	73
11.4.1	Three-level scheduling	74
11.5	Scheduling Example	74
11.6	Minix IPC	75
11.7	Minix structures	75
11.8	Where do we go from here?	75

12 Lecture: MINIX IO Architecture	76
12.1 Announcements	76
12.2 Where do we go from here?	76
12.3 Input/Output	76
12.4 Devices	77
12.5 Deadlock and its avoidance	77
12.5.1 Deadlock Avoidance Methods	78
12.6 Banker's Algorithm (Dijkstra, 1965)	78
12.6.1 Single Resource	78
13 Lecture: Minix IO, cont.	81
13.1 Announcements	81
13.2 Managing Multiple Resources	81
13.2.1 Resource Trajectories	82
13.3 Managing Multiple Resources, cont.	82
13.3.1 Multi-way bankers'	82
13.4 Wrapping up deadlock avoidance	83
13.5 Back to I/O	83
13.6 Devices	83
13.6.1 Low level considerations: timing, interleaving, etc.	83
13.6.2 Accessing a device: Device Controllers	83
13.7 Reading from a device: DMA vs. Programmed IO	85
14 Lecture: IO Software	86
14.1 Announcements	86
14.2 Back to I/O	87
14.3 Devices	87
14.3.1 Low level considerations: timing, interleaving, etc.	87
14.3.2 Accessing a device: Device Controllers	87
14.4 Reading from a device: DMA vs. Programmed IO	87
14.5 Goals of IO Software	88
14.6 IO Software	88
14.7 Minix IO Structure	88
14.8 Minix IPC	88
14.9 Structure of Minix: Client/Server model	89
14.9.1 Interrupt Handlers	89
14.9.2 Simple	89
14.9.3 complex	89
14.10 Structure of Minix: Device Tasks (Drivers)	90
14.11 Driver Behavior	90
14.11.1 Example: Disk seek optimization	90

List of Figures

1	A Linux <code>open()</code> implementation	28
2	Minix <code>open()</code> implementation	29
3	Minix <code>_syscall()</code>	29
4	Minix <code>sendrec()</code> abstracted	30
5	The process of a context switch	35
6	Possible states for a process	37
7	Possible states for a process	40
8	The lowest layer of the operating system	40
9	Structure of VM/370	43
10	Layout of the Minix system	44
11	Stack development (Remember that the real stack is upside-down)	50
12	Peterson’s solution for mutual exclusion	57
13	A producer-consumer implementation with a race condition	60
14	A semaphore-based producer-consumer implementation	61
15	A monitor-based solution to the producer-consumer problem.	63
16	A message-passing solution to the producer-consumer problem.	66
17	Possible states for a process	69
18	Layout of the Minix system	73
19	How a deadlock forms	78
20	One dimensional example of Dijkstra’s Bankers Algorithm	79
21	Another example, but one that doesn’t wedge immediately.	80
22	Resource trajectory example from Tanenbaum	82
23	Multi-dimensional Bankers Algorithm	84
24	Minix IO	88

List of Tables

1	Integer registers of the x86 CPU	48
---	--	----

0 Syllabus

cpe453 — Introduction to Operating Systems

Sections 01 and 03 (Nico)

Spring 2026

Administrivia

Professor: Dr. Phillip Nico
pnico@calpoly.edu

Office Hours: (subject to change)
Monday: 2:10pm–3:00pm¹
Tuesday: 12:10pm–1:00pm¹
Wednesday: 2:10pm–3:00pm¹
Thursday: —
Friday: 2:10pm–3:00pm¹
or by appointment

Texts: Andrew S. Tanenbaum and Albert S. Woodhull. *Operating Systems: Design and Implementation*, Third edition, Prentice-Hall, 2006.

You might also find the following books to be valuable resources:

Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, Second edition, Prentice-Hall, 1989.

W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, The Addison-Wesley Professional Computing Series, Addison-Wesley.

Webpage: <http://www.csc.calpoly.edu/~pnico/class/now/cpe453>

Lecture: Section 1: MWF 9:10am–10:00am 22-0212
Section 3: MWF 11:10am–noon 03-204

Lab: Section 2: MWF 10:10pm–11:00pm 14-302
Section 4: MWF 12:10pm–1:00pm 14-301

Midterm: Wednesday, May 6th (subject to change)

Final²: (Unreliable. Verify with the schedule of classes.)
Section 1: Wednesday, June 10th 7:10am–10:00am
Section 3: Monday, June 8th 10:10am–1:00pm

Grading: The approximate grading breakdown (subject to change) is:
Laboratory Exercises 15%
Programming Assignments 25%
Midterm 25%
Final 35%

¹Office hours are guaranteed until the earlier of the posted end time or the time at which there are no more students. Beyond that, I won't go far, but I might be "around."

²There is a chance that the final will be moved to one of the common final times this quarter.

Course Objectives

The purpose of this course is to gain experience with and understanding of operating systems principles and implementation. In the process, you will:

- Examine the requirements of a modern operating system, including the fundamental problems of managing concurrent processes.
- Understand the system call interface to an operating system.
- Understand how an operating system gets started (boots) and takes control of the machine.
- Understand the design and implementation of a filesystem.
- Learn a bunch of other interesting things.

Prerequisites

The prerequisites for this course are cpe 225 or cpe 233 and cpe 357. Cpe 453 assumes proficiency at writing programs in the C language and understanding of the compilation, assembly, linking and loading processes, as well as understanding of contemporary computer architecture at the instruction set level.

If you have any doubts, come talk to me.

Course Format

The course consists of three lectures and three labs a week. The labs will not meet formally every time—possibly not ever—but I reserve the right to call lab meetings for demos or exercises, etc. In general, I will introduce new material in the lectures, and the laboratory exercises will consist of practical application of the material. However, it is entirely possible that I will introduce some new material in lab.

The purpose of the class is for everyone to understand about operating systems and their implementation. To this end, if you don't understand something during class, ask. If you are confused, it is likely that a few dozen of your classmates are as well. Also, listen to others' questions. Many times you'll think you understand a concept until you hear someone else's question about it. Dialogue is the best way to learn things, so don't be afraid to speak up.

Office Hours

Office hours are as listed above **or by appointment**. If you are unable to attend the posted office hours, contact me and we can arrange something. There is no reason why any of you should be unable to see me if you need to.

Other Resources

I will maintain a class web page at <http://www.csc.calpoly.edu/~pnico/class/now/cpe453>. On it I will keep information, assignments, announcements, etc. If there any class announcements, corrections, etc., to be made, I will post them in the Announcements section of the class web page. Please check the web page regularly. I will try to make any announcements in class as well, but I cannot guarantee it, and you don't want to miss anything.

I usually post my lecture notes on the web. These are guaranteed to be incomplete and are not a substitute for class attendance. They will, I hope, provide a framework for your own notes and emphasize what I think is important about the class.

Depending on how much time I have, I will also maintain a FAQ on the class page of questions from office hours that seem to be of general interest.

Laboratory Exercises and Programming Assignments

Programs:

There will be a number of assignments over the quarter. These will consist of some combination of programming exercises and reports on system modifications. Together they will represent 25% of the total final grade for the course. Because the total number is not known at this point, the individual program weights will not be determined until the end of the quarter. I expect there will be around 5 assignments over the quarter, but this may vary.

Programming assignments will be distributed on the web, and each assignment will specify both when it is due and whether or not partnerships are allowed.

Labs/Problem Sets:

Most weeks of the quarter there will be a set of laboratory exercises intended to supplement the coursework. These will consist of written exercises and/or experimental work to be performed in the lab. Submission requirements will be posted with each lab.

Late Policy:

Each student will be allowed three (3) discretionary late days to be applied over the quarter. One late day will be required for each calendar day (not work day) or portion thereof after the due date. To submit work after the submission directory for an assignment has been closed, use the `latedays` program to reopen it for you. Instructions will be provided on the class web site.

Note: There may be some assignments for which the use of late days will not be allowed. This would be to facilitate the posting of solutions before the midterm or some other reason like that. It will be noted on these assignments that late days are not permitted.

If you are unable to complete an assignment by the specified time and do not have any more late days, turn in what you have for partial credit. **Late assignments will receive no credit.**

Partnerships:

On some of the assignments you may be working with partners. Collaboration tends to help with figuring out difficult concepts and generally makes the whole process more pleasant. A word of caution, though: While it is tempting to just divide up the work, be sure each partner understands the whole project. Concepts learned on the assignments will show up on exams which are worth far more than the individual assignments in the final analysis. Even if your partner bails you out of a tight spot, be sure you understand the work, or it will come back to haunt you. Be absolutely certain that both partners' names appear on all assignments. Credit will be given only to students whose names appear on the assignment.

Each assignment will specify whether working with a partner is permitted.

Submitting Written Work:

Written work should be submitted on the day due (as modified by late days). Clearly nobody will actually be submitting written work this quarter, but on the off chance someone hand writes work and scans it, here is a modified set of my usual rules:

For all written work, in order to facilitate grading:

- Start a new page for a new problem if you will not be able to complete it on the current page.
- Place the problems in the order assigned.

- Do not write in red.
- Use proper English grammar and punctuation.
- Write legibly.

Neatness will not necessarily help you, but sloppiness will definitely hurt. If I can't read it, I can't grade it, and I will not guess at what you meant. (This also applies to exams.)

These rules may seem extreme, but they are intended to make grading easier so I can return your papers more quickly.

Submitting Programs:

Programming assignments will be submitted online on the Computer Science Department CSL machines using the **handin** program. Instructions for submitting programs will be provided on the class web page.

When turning in programming assignments, be careful to submit your final version and to have tested it before submitting.

Programs that fail to compile will receive no credit.

Exams

There will be a midterm and a final. The midterm will be worth 25% of the final course grade and the final will be worth 35%. Exams will emphasize insight and problem solving ability rather than memorization.

Missed Exams: Makeup exams will only be given for the gravest of reasons. If you must miss an exam due to extreme illness, etc., contact the instructor (phone or email is fine) or leave a message with the Department of Computer Science office (805-756-2824) *before* the exam. Be sure to leave both the reason for missing the exam and how to reach you.

Grading Policy

The final grade for this course will be constructed of three components in the following proportions:

Laboratory Exercises	15%
Programming Assignments	25%
Midterm	25%
Final	35%

In general, letter grades will be computed according to the scale:

Minimum A–:	90%
Minimum B–:	80%
Minimum C–:	70%
Minimum D–:	60%

That is, if you score 90% on an exam you will be guaranteed at least an A– for that exam. On some assignments and exams the minimum cutoffs may (and probably will) be lowered to account for difficulty.

Note: I reserve the right to take other circumstances into account when assigning final grades. These include, but are not limited to, such things as substantial improvement over the quarter, significant differences between exam and homework performance, missing homeworks, etc.

Nothing would make me happier than to give everyone an A.

Missing Assignments: Unless special arrangements have been made in advance, a good-faith effort is required of each student for every program. Failure to submit a homework disqualifies the

student from the top grade level possible for a final grade. Failure to submit two disqualifies from the top two levels, etc.

Regrades: In general, papers to be considered for regrades must be submitted within one week of the time they become available for picking up. The same is true for misrecorded grades. They must be reported within a week of their posting. Grade feedback will be mailed out periodically as things are graded, and a snapshot of my gradebook for each of you will be available linked from the class web page. Please check them to be sure they agree with your own records.

Collaboration and Cheating

Policy on Collaboration

Programming assignments in this class are intended to be demonstrations of individual or partnership abilities. To this end, programs are to be written only by the designated authors.

High-level discussion of problems and problem-solving techniques, however, is beneficial to all involved. You are encouraged to discuss approaches so long as those with whom you consult are given due credit in your program headers.

It is *never* acceptable to allow someone else to have your work for reference or to refer to someone else's work while writing your own.

In this case, “someone else’s work” means not only other students’ programs, but also materials from any other source, including, but not limited to, the Internet, other reference books, etc., *including the use of generative AI*. Also, “not giving your work to others” includes taking reasonable precautions to prevent them from taking it.

Collaboration that goes beyond general approaches or that is uncredited will be considered cheating. If you are unsure about what constitutes proper or improper collaboration, consult the instructor for guidance.

Policy on Cheating

Don’t. I consider academic dishonesty a serious offense. Any instances of cheating or plagiarism will be referred to the Office of Student Rights and Responsibilities. The Cal Poly rules and policies are listed in the catalog as well as at the OSRR web site, <http://www.osrr.calpoly.edu>. The general policy, however, is very simply stated in the Campus Administrative Manual (C.A.M. 684):

Cheating requires an “F” course grade

Turning in work is presumed to be a claim of authorship unless explicitly stated otherwise.

If the course rules are unclear or you are unsure of how they apply, ask the instructor *beforehand*.

Feedback

One of the frustrations of teaching is that the instructor rarely gets any feedback on the course until the teaching evaluations at the very end when it is too late to do anything about it. If you like, dislike, or don’t understand something I’m doing with the course, please stop by my office hours, send me email, or paste together a note from newspaper clippings and drop it in my mailbox. I won’t always change things, but I will always explain why I’m doing them the way I am.

Tentative class schedule

The tentative schedule for the course is given on page 8. This is what we hope to accomplish and when we hope to accomplish it. There may be changes, but this is a rough roadmap for the quarter.

Spring 2026 Preliminary Course Outline

(Subject to Change)

Week	Lecture	Topic	Reading	Date
1	1	Introduction and Background	Chapter 1	March 30
	2	History		April 1
	3	The Process Model	Chapter 2	April 3
2	4	Processes, cont.		April 6
	5	Operating System Structures		April 8
	6	Concurrency		April 10 ²
3	7	Concurrency, cont.		April 13
	8	Even More Concurrency		April 15
	9	Wrapping up Concurrency		April 17
4	10	Scheduling	Chapter 3	April 20
	11	Input and Output		April 22
	12	Device Management		April 24
5	13	IO Systems and Deadlock		April 27
	14	IO Systems, cont.		April 29
	15	MINIX IO Architecture		May 1
6	16	Memory Systems	Chapter 4	May 4
	17	Midterm		May 6
	18	Dynamic Segmentation		May 8
7	19	Virtual Memory		May 11
	20	Even More Memory		May 13
	21	Page Replacement Algorithms		May 15
8	22	Virtual Memory Wrapup		May 18
	23	Filesystems	Chapter 5	May 20
	24	Filesystems, cont.		May 22
9	—	<i>Memorial Day</i>		May 25
	25	Directory Schemes		May 27
	26	Index Schemes		May 29
10	27	Other filesystems		June 1
	28	Wrapping up Filesystems		June 3
	29	T.B.A		June 5
Final Exam		Section 1: Wednesday, June 10th, 7:10am–10:00am Section 3: Monday, June 8th, 10:10am–1:00pm <i>(Verify with published schedule)</i>		

²Last day to drop classes is Thursday, April 9.

The Last Page

In a normal term I'd have you fill out this last page and turn it in. This is not a normal term, so this will be replaced by lab 0. See the class web page.

This page is so I can gather a little information about you at the beginning of the class. Please fill it out, tear it off and leave it with me on the way out.

Who are you?

Name: _____
Section: _____
Major: _____
Email: _____

Class Expectations?

Please take a minute to write out what your goals and expectations are for cpe 453. What do you want to learn?

I hate to do this, but to be sure there's no confusion on the matter...

Below, please copy the two boxed text segments from page 5 about academic dishonesty and sign the pledge (assuming you will comply, of course). Without this you will automatically receive a grade of zero for all assignments.

1)

2)

Pledge

I will do my own work in this class. That is, unless it is explicitly permitted by the assignment, I will neither use others' work as my own nor make my work available for others to use. I understand that either of these actions constitutes cheating sufficient to merit a grade of F for the course.

Signature

Date

1 Lecture: Introduction and Background

Outline:

- Robust Programming
- Background: The Ballad of the Unknown Stuntman
- Syllabus
- Preparation
- How to succeed in this course
- The Last Page
- Foundations
 - What an operating system is
 - Virtualization and Transparency
 - What an operating system is not
- Major subjects for the quarter
- This quarter
- About the lab
- About the assignment

- Check on the book. (Third edition, Racoons)
- Make sure your lab accounts work.
- Assignments and labs are out
 - Coming attractions:

Event	Subject	Due Date		Notes
asgn3	dine	Fri	May 1	23:59
lab03	problem set	Mon	May 4	23:59
midterm	stuff	Wed	May 6	
lab04	scavenger hunt II	Wed	May 13	23:59
asgn4	/dev/secret	Fri	May 22	23:59
lab05	problem set	Mon	Jun 1	23:59
asgn5	minget and minls	Wed	Jun 3	23:59
asgn6	Yes, really	Fri	Jun 5	23:59

Use your own discretion with respect to timing/due dates.

- Office hours
 - Come
 - With high probabability they are:
 - Monday: 2:10pm–3:00pm¹
 - Tuesday: 12:10pm–1:00pm¹
 - Wednesday: 2:10pm–3:00pm¹
 - Thursday: —
 - Friday: 2:10pm–3:00pm¹
 - These may be changed as things get sorted out.
 - Also lab times

¹Office hours are guaranteed until the earlier of the posted end time or the time at which there are no more students.

- If you’ve been through this before unsuccessfully, *tell me*. I can ask you awkward questions that’ll improve your odds of never doing 453 again.
- Forum (linked from web page)
- A note on persistence (tortillas?)
- Expect to fail. A lot.

1.1 Robust Programming

Do it.

1.2 Background: The Ballad of the Unknown Stuntman

Usually—if all is going well, that is—we aren’t even aware of the operating system. The operating system of a computer is the group of stagehands making everything work. Without them, the show couldn’t go on, but you only notice if something doesn’t work.

“The OS is our enemy?”

Have you ever wondered why your PC won’t speak to your printer even though the printer seems to be being extremely polite?

1.3 Syllabus

- The Course Staff
 - me: Phillip Nico
pnico@calpoly.edu
Office: 14-205

- About this course:

What is this course about?

Why Study OS?

- Because it explains the magic

What does that mean?

- OS Theory:
 - * Concurrent programming (deadlock avoidance)
 - * Scheduling algorithms (fairness, starvation)
 - * Resource allocation (e.g., Memory allocation (paging) algorithms)
- OS Implementation: How it’s really done (The lab)
- Stark Raving Paranoia: The OS can’t stop, can it?
- Course Objectives

The purpose of this course is to gain experience with and understanding of operating systems principles and implementation. In the process, you will:

- * Examine the requirements of a modern operating system, including the fundamental problems of managing concurrent processes.

- * Understand the system call interface to an operating system.
- * Understand how an operating system gets started (boots) and takes control of the machine.
- * Understand the design and implementation of a filesystem.
- * Learn a bunch of other interesting things.

- Prerequisites:

203 and (225 or 233) Know how to program and understand architecture.

cpe 357 Be familiar with the C programming language, user-level unix, and the “user-side” of system calls.

- Prerequisites. You **must** have satisfied the **prerequisites**.

If you have not, drop the class. (Seriously.)

- Re-takes

- If you are re-taking, perhaps talk to me.

- Enrollment:

Current (out of date):

	sec 01	sec 03
enrolled	30	30
target	30	30
waitlist	10	10
unique	20	

We’re gonna try something crazy and let the registration system do its thing, then talk about it.

- Web Site: <http://www.csc.calpoly.edu/~pnico/class/now/cpe453>

Just about everything of interest ends up here, including:

- The syllabus (that you should read)
- Notes
- Assignments (read these too. It helps.)
- Labs
- Solutions
- (Probably) grades

- Texts: Andrew S. Tanenbaum and Albert S. Woodhull. *Operating Systems: Design and Implementation*, third edition, Prentice-Hall, 2006. ISBN 0-13-638677-6

If you are unfamiliar with the UNIX system programming environment, you might also find the following helpful:

- W. Richard Stevens, *Advanced Programming in the UNIX Environment*, The Addison-Wesley Professional Computing Series, 1/e, Addison-Wesley, 1992.

- Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, Third edition, Prentice-Hall, 2006.
- Also: The Minix Programmer’s Manual (online—use the version on your machine)
Note: “`cmd(num)`” means `cmd` as defined in section `num` of the manual.
- Grading policy:

Labs	15%
Assignments	25%
Midterm	25%
Final	35%
- Assignments: (**Do Them**)
 There will be some. A variety of programming, problem set, and report writing, totaling 25% of the final grade.
 I expect there will be 5–6 assignments over the quarter, making each one worth *approximately* 5%
 I will publish weights, but reserve the right to change them.
 Asgn1 is out already. **GET YOUR ACCOUNTS NOW** you can even use the lab time.
 Deliverable: a program or a report.
Note on assignments:
 - Do them!
 - Try to break them
 - * with real tests
 - * on multiple platforms (e.g. 32- and 64-bit)
 - If I provide a test harness, run it. It may not be my entire grading rig, but it’s probably a subset. (And it might be.). These are located in `~pn-cs453/demos`, usually named `tryAsgnN` or `tryLabN`.
 Don’t copy it. Run it from there. (Remember, to run something you name it.)
 - RTFA
- Labs (6–7):
 Most weeks there will be lab exercises. These will be some combination of written problems and laboratory exercises.
 Deliverable: usually a report, sometimes a program.
 A note on the lab period.
- Policies regarding submission of work:
 - Late Policy
 - Submitting Work (paper(not this quarter), online (I’ll check on this)) neatness.
 - Late days

- “Good faith effort” required on all assignments. (Missing assignment reduces maximum possible grade.)
- Exams: a midterm and a final. Emphasis on problem solving.
The midterm is tentatively scheduled for May 7
The final? We’ll see.
- Grading: Policy. 1-week deadline for regrades/errors.
- Class decorum
 - I encourage interaction in class. If you’re confused, so are other people.
 - Note
 - * you all belong here.
 - * Remember: of course you don’t know what you are doing. That’s why you’re in the class. Speak up.
 - I will not penalize you for nonattendance, but you will lose sympathy in office hours. (Find out what happened)
 - Please refrain from disruptive behavior.
 - I try to keep the two classes in sync, but I cannot guarantee it. I do ask students to attend their own sections for exams.
- Cheating: Don’t.
 - Spring quarter was a horrorshow
 - Integrity matters
 - Partnerships and proper collaboration.
 - * DO: use each other for help
 - * DO NOT: simply split up the work
 - Proper Collaboration
 - * general principles/approaches
 - * debugging consultation
 - Improper Collaboration
 - * “Here’s my code”
 - * *specific* solutions
 - Apply the “Competition” rule
 - Credit any work that comes from anyplace else. (Not doing your homework is better than getting caught cheating. Your name isn’t Tanenbaum, is it?)

Thought:

- If the only penalty is a bad grade...

The Pledge:

- Don't use others' work while doing your own
- Don't make your work available. *This includes taking reasonable precautions to prevent someone else's taking it.* E.g., not leaving your program lying around or leaving it online world readable.

It is *never* acceptable to allow someone else to have your source code for reference or to refer to someone else's code while writing your own.

Cheating requires an "F" course grade

Not giving your work to others includes taking reasonable precautions to prevent others taking it.

1.4 How to succeed in this course

- Read assignments right away so they can be percolating.
- Do discuss problems with classmates and/or me
- Give yourself time to be stuck
- "I need insight now" is not effective
- When I give test harnesses, use them:
 - tryLab01 (*don't copy it*)
 - tryAsgn1 (*don't copy it*)
- It sounds silly to say, but: **turn stuff in.** (You can't win if you don't play).
- Read the textbook. Tanenbaum is a good writer.

1.5 The Last Page

Please fill this out as it helps me to know who you are, where you're coming from, and what you expect from the course. It also helps me gauge enrollment.

1.6 Foundations

1.6.1 What an operating system is

Virtual Machine (Extended (abstract) machine) Simplified view of the system.

Operating Systems insulate users from the complexity of the underlying system and provide functionality that doesn't exist.

(e.g. a filesystem: do you *really* care how a file is stored?)

(Do you want to know (or care) whether the floppy motor is on? The OS does. (and you do. You just want someone else to take care of it.))

Resource Manager Combination mediator and traffic cop.

(Consider printer spooling.)

Typically this is possible because only the OS has access to certain instructions. (kernel or supervisor mode)

This is just protecting from accidents, let alone malice.

The operating system is the one who *really knows what's going on*. (and makes the magic happen)

That's all well and good, but what *is* an operating system?

a program. That's it.¹ Rather, it's the first program that allows others to run. The things above are what makes an OS *useful*.

The OS can be considered the host of the party: it's the first to arrive, the last to leave (after cleaning up), and mostly it tries to stay out of the way and keep the party going, but steps in where necessary to resolve conflict.

1.6.2 Virtualization and Transparency

From a user's perspective this process reduces to *virtualization* and *transparency*

transparent It's there, but you can't see it.

virtual you can see it, but it isn't there.

1.6.3 What an operating system is not

- Compilers
- Editors
- Command interpreters
- **Web Browsers(!)**

Compilers	Editors	Shells
Operating System		
Machine Instructions		
Physical Hardware		

1.7 Major subjects for the quarter

Three areas I really want to develop this quarter:

1. OS Theory: How can we solve various problems well?
 - Concurrent programming (deadlock avoidance)
 - Page algorithms
 - Scheduling algorithms (fairness, starvation)
2. OS Implementation: How it's really done (The lab)
3. Stark Raving Paranoia: The OS can't stop, can it?

¹This answer would be considered correct, but not complete were I to ask the same question on a midterm. :-)

1.8 This quarter

What we're looking at:

- How the OS does its magic
General areas:
 - Processes and Concurrency
 - IO
 - Memory Management
 - Filesystems
 - Security?
- This class:
 - Why study OS?
 - Why Unix? — innards are exposed
 - Why Minix? — there aren't too many innards
 - * only 30,000 lines of code (vs. ≈ 9 million for Linux)
- Resources:
 - the CSL machines

1.9 About the lab

Remember `man name(section)`

- `pipeit (ls | sort -r > outfile)`

What does this mean?

- There are three processes here
- You are the plumber/reaper
- This demonstrates the process abstraction that an OS provides. All communication and synchronization takes place through the OS.
- These must be concurrent (why?)

1.10 About the assignment

- `malloc(3)`: How does it work?
- libraries: Two forms
 - static (`libmalloc.a`)
 - shared object (`libmalloc.so`)
 - * `LD_LIBRARY_PATH`
 - * `LD_PRELOAD`

- don't call `sbrk(2)` for every call to `malloc(3)` (quilting analogy)
- remember how pointer arithmetic works (in the size of the pointee)
- `uintptr_t` from `<stdint.h>`
- About that Makefile...
- Note: the order of link commands matters to `gcc`

- Also Note: setting environment variables:

[ba]sh	<code>VAR=value</code>
	<code>export VAR</code>
[t]csh	<code>setenv VAR value</code>

2 Lecture: History and Definition of an O.S.

Outline:

- Announcements
- About the lab
- About the assignment
- Everything you wanted to know about C development
- Aside: Review of 357
 - Review of Unix IO
 - Processes, etc.
 - Compilation

2.1 Announcements

- Be reading along in the book. Tanenbaun and Woodhull are good writers.
- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- Late Days (how they work)
- Beware of malloc(): there are a kajillion implementations out there, but you need to do your own.
- If you've been through this before unsuccessfully, *tell me*. I can ask you awkward questions that'll improve your odds of never doing 453 again.
- tryLab01
 - `~pn-cs453/bin/longlines.pl`
- tryAsgn1
 - (*don't copy it*)
 - Run it on a 64-bit machine (e.g. unix1–4)
(Make sure it's one with 32-bit libraries; unix5 doesn't have them for whatever reason.)
 - Consider the effects of architecture (how big is an int?) and uninitialized data (arm.csc.calpoly.edu f35.csc.calpoly.edu)
 - These are shell scripts. You can read them
- Office hours

- Come
- Or talk to me during lab. I'm guaranteed to be available.
- “<https://www.cs.vu.nl/~ast/intel/>”
- `gdb` and `valgrind` are your friends.

2.2 About the lab

Remember `man name(section)`

- `pipeit (ls | sort -r > outfile)`

What does this mean?

- There are three processes here
- You are the plumber/reaper
- This demonstrates the process abstraction that an OS provides. All communication and synchronization takes place through the OS.
- These must be concurrent (why?)

2.3 About the assignment

- `malloc(3)`: How does it work?
- libraries: Two forms
 - static (`libmalloc.a`)
 - shared object (`libmalloc.so`)
 - * `LD_LIBRARY_PATH`
 - * `LD_PRELOAD`
- don't call `sbrk(2)` for every call to `malloc(3)` (quilting analogy)
- remember how pointer arithmetic works (in the size of the pointee)
- `uintptr_t` from `<stdint.h>`
- About that Makefile...
- Note: the order of link commands matters to `gcc`

- Also Note: setting environment variables:

[ba]sh	<code>VAR=value</code>
	<code>export VAR</code>
[t]csh	<code>setenv VAR value</code>

2.4 Everything you wanted to know about C development

- The Environment
- Linking
- Loading
- Make
- gdb

2.5 Aside: Review of 357

2.5.1 Review of Unix IO

- file descriptors
- `open(2)` vs. `fopen(3)` (and permissions)
- `dup(2)` and `dup2(int old, int new)`
- pipes, how they work

2.5.2 Processes, etc.

- Lifecycle

Birth	<code>fork()</code>
Death	termination (<code>exit()</code> , <code>_exit()</code> , <code>return</code> , <code>abort()</code> , <code>signal</code>)
Afterlife	reaping with <code>wait()</code> or <code>waitpid()</code>

 - `wait(2)`
 - `waitpid(2)`
 - `WIFEXITED()`/`WIFSIGNALED()`/
 - `WEXITSTATUS()`

2.5.3 Compilation

- The compiler (`gcc`, `ACK`, `clang`)
- The linker (`ld`, `gcc`)
 - `-Lpath`
 - `-lname`
 - `LD_LIBRARY_PATH`
 - `LD_PRELOAD`
- The loader (`ld.so`)
- Libraries
 - Static (`*.a`). Made with `ar(1)`
 - Dynamic (`*.so`, `*.dll`, `*.dylib`) Made with the compiler
- Some thoughts on Make

3 Lecture: History

Outline:

- Announcements
- Defining of an Operating System: The System Calls
- Two stories
- OS History
 - Ancient times
 - The middle ages
 - The renaissance: families
 - Modern Times
- Defining of an Operating System: The System Calls
- System Calls Again
- System Call Mechanisms
 - How to do it
- OS Pre-history: The boot process
 - How it all begins (on an Intel PC with a floppy)
 - How it continues
 - And onwards: How does the OS get control back?
 - Tool of the week: Make(1)

3.1 Announcements

- Asgn 1 due Wednesday
- For Asgn1:
 - don't call `sbrk(2)` for every `malloc(3)` call.
(quilting?)
 - remember how pointer arithmetic works (in the size of the pointee)
 - `uintptr_t` from `<stdint.h>`
 - About that Makefile...
 - `free(NULL)` called by `snprintf(3)`. Watch for overflows
 - (also, `-m32` and `intel-all`)
 - Compiling on 32- vs. 64-bit architectures.
 - (and `printf(3)`, etc, might use `malloc(2)`. Consider `puts(3)` and/or `write(2)`.)
- tryAsgn1
 - How to do alignment? Arithmetic: `stdint.h` and, esp., `uintptr_t`
 - `free(NULL)` called by `snprintf(3)`. Watch for overflows
 - (also, `-m32` and `intel-all`)
 - How to use late days
 - Don't leave things until the last minute
 - Path to tests can be discovered ($n/2 + 1$)

- lab02 out.
- Coming attractions:

Event	Subject	Due Date		Notes
asgn3	dine	Fri	May 1	23:59
lab03	problem set	Mon	May 4	23:59
midterm	stuff	Wed	May 6	
lab04	scavenger hunt II	Wed	May 13	23:59
asgn4	/dev/secret	Fri	May 22	23:59
lab05	problem set	Mon	Jun 1	23:59
asgn5	minget and minls	Wed	Jun 3	23:59
asgn6	Yes, really	Fri	Jun 5	23:59

Use your own discretion with respect to timing/due dates.

3.2 OS History

The history of the development of operating systems parallels the history of computer architecture. (duh?)

3.2.1 Ancient times

Vacuum tubes/relays. No programming languages. Bug meant an actual insect. (aside about Dijkstra: (paraphrased) we should eliminate the use of the word *bug* and replace it with *error*.).

“Human operating systems” rewire the machines

- programmed by the builders.
- time blocked out for exclusive access
- machines rewired with plugboards
- Eventually evolved into card-fed computers.

(*library* actually meant a filing cabinet)

3.2.2 The middle ages

Transistors made machines small and reliable. OS loaded a program, then replaced itself.

Batch processing invented (IBM 1401→7094→1401)

- jobs loaded onto a tape offline
- run in the main computer
- output printed offline
- operations controlled by JCL

3.2.3 The renaissance: families

Computer *families* developed. Different machines of different sizes. (e.g., IBM System/360 note Brooks' experience)

operating systems very complex, written in assembly, tied to a particular processor (family).

Innovations:

- multiprogramming (but still essentially batch systems)
- timesharing (MULTICS: Multiplexed Information and Computing Service)
- the rise of the minicomputer! (PDP-1 1961, \$120,000² vs. \$2.4M³ for a 7094).
- a neglected PDP-7 and Unix: (1969)
 - History
 - * Space Travel (Thompson and Ritchie)
 - * Small system
 - features
 - * Small system
 - * modular
 - * portable (Written in C!)
 - BCPL (untyped (all datatypes are bitfields))
 - B (untyped)
 - C (weakly typed)
 - Ansi C (more strongly typed) – 1989

3.2.4 Modern Times

In modern days, the rise of the PC parallels the rise of the minicomputer:

- Windows/unix (Tanenbaum vs. Torvalds?)

The family tree:

- AT&T
- BSD (Berkeley Software Distributions)
- Linux?
- POSIX 1988

- Where does minix fit in this?

Minix vs. Linux? A matter of Philosophy

- Minix
 - Designed for readability

²According to the CPI, this is \$1 267 235 in 2024

³\$25 352 698 in 2024

- Modularity
- Message passing
- extensibility
- linux
 - Efficiency
 - “Production” system

3.3 Defining of an Operating System: The System Calls

Tanenbaum set out to write a UNIX... what does that mean?

What defines an operating system from the users’ point of view?

The system calls.

Just as the instruction set architecture defines an architecture, the system calls define an Operating System.

Before we talk about IO services, implementation, etc....

Look like C functions, but are direct requests for OS services.

A system call is an entry into the kernel.

Linux: (RH7.0) 222
Solaris: 253
Minix: 53

3.4 System Calls Again

We said “An OS is defined by its system calls”. What does that mean?

System Call

A system call is the means by which the kernel provides access to a particular operating system service, and the services available through system calls (e.g., reading and writing the disk, allocating memory, starting new processes, etc.) are reserved to the kernel; there is no other way of doing these things.

3.5 System Call Mechanisms

Last time we said

- It’s all about privilege.

3.5.1 How to do it

How done? Machine dependent. usually in assembly, but hidden from the users’ view.

In any case, some sort of trap is involved to get supervisor privileges.

A *trap* is a software generated interrupt. Exactly what happens varies from architecture to architecture, but the result usually involves:

- saving the state of the currently executing program

- elevating of processor privilege level to “supervisor”
- transfer of control to a pre-registered routine
- *The OS does something*
- Then, usually:
 - privilege level is reduced back to user
 - original state of executing program is restored

3.6 OS Pre-history: The boot process

3.6.1 How it all begins (on an Intel PC with a floppy)

- Power on
- Boot Sector of fd0: 512b⁴, loaded into 0x00007c00,
- jump to 0x00007c00...

3.6.2 How it continues

- OS Sets up whatever it needs to set up, and, depending on what sort of system it is...
- (maybe) picks a program to run
- (maybe) shifts out of supervisor mode
- (maybe) runs the program

3.6.3 And onwards: How does the OS get control back?

How does the OS preempt a user process? It’s not running.

It takes a series of carefully-planned steps

- The O.S. runs in supervisor mode which allows it to manipulate the interrupt vector (definition) and register an ISR.
- Before changing privilege levels, the O.S.
 - installs ISRs for a timer and also for the system call interrupt, and
 - requests a timer interrupt for some future time.
- The O.S. then changes privilege levels and yields to the user process.
- Eventually, one of two things happens:
 1. the timer interrupts, or
 2. the process makes a system call

⁴1440k floppy has 2 sides, 80 tracks, 18 sectors. This makes 512 bytes per sector. $(1474560/80)/18 \rightarrow 18432/18 \rightarrow 512$

```

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdarg.h>
#include <errno.h>

#define SYSCALL_OPEN 0x5

/* make sure this is legitimate */
#ifndef _i386
#error "This can only be compiled for an x86"
#endif

int open(const char *pathname, int flags, ... ) {
    mode_t mode;
    va_list ap;
    int res;

    /* use the variable arguments support */
    va_start(ap, flags);
    mode = va_arg(ap, mode_t); /* extract the mode from the top of the stack */
    va_end(ap);

    /* load the given values into the registers and execute
     * the given syscall. The syscall code goes into eax
     * and return value comes from there.
     */

    asm ("movl %0,%%eax" : : "g" (SYSCALL_OPEN) : "eax");
    asm ("movl %0,%%ebx" : : "g" (pathname) : "ebx");
    asm ("movl %0,%%ecx" : : "g" (flags) : "ecx");
    asm ("movl %0,%%edx" : : "g" (mode) : "edx");
    asm ("int $0x80" : : : "eax");
    asm ("movl %%eax,%0" : "=g" (res) :);

    if ( res < 0 ) {
        errno = -res;
        res = -1;
    }

    return res;
}

```

Figure 1: A Linux `open()` implementation

```

/* ./lib/posix/_open.c */
#include <lib.h>
#include <fcntl.h>
#include <stdarg.h>
#include <string.h>

PUBLIC int open(const char *name, int flags, ...)
{
    va_list argp;
    message m;

    va_start(argp, flags);
    if (flags & O_CREAT) {
        m.m1_i1 = strlen(name) + 1;
        m.m1_i2 = flags;
        m.m1_i3 = va_arg(argp, Mode_t);
        m.m1_p1 = (char *) name;
    } else {
        _loadname(name, &m);
        m.m3_i2 = flags;
    }
    va_end(argp);
    return (_syscall(FS, OPEN, &m));
}

```

Figure 2: Minix `open()` implementation

```

/***** lib/other/syscall.c *****/
PUBLIC int _syscall(who, syscallnr, msgptr)
int who;
int syscallnr;
register message *msgptr;
{
    int status;

    msgptr->m_type = syscallnr;
    status = _sendrec(who, msgptr);
    if (status != 0) { /* 'sendrec' itself failed. */
        msgptr->m_type = status;
    }
    if (msgptr->m_type < 0) {
        errno = -msgptr->m_type;
        return(-1);
    }
    return(msgptr->m_type);
}

```

Figure 3: Minix `_syscall()`

```

/*****./lib/i386/rts/_sendrec.s*****/
! See ../h/com.h for C definitions
SEND = 1
RECEIVE = 2
BOTH = 3
SYSVEC = 33

SRCDEST = 8
MESSAGE = 12
!*=====
!                                     _send and _receive *
!*=====
! _sendrec() save ebp, but destroy eax and ecx.
.define _sendrec
.sect .text

_sendrec:
    push    ebp
    mov     ebp, esp
    push    ebx
    mov     eax, SRCDEST(ebp)    ! eax = dest-src
    mov     ebx, MESSAGE(ebp)    ! ebx = message pointer
    mov     ecx, BOTH           ! _sendrec(srcdest, ptr)
    int     SYSVEC              ! trap to the kernel
    pop     ebx
    pop     ebp
    ret

```

Figure 4: Minix `sendrec()` abstracted

Either one causes an interrupt.

- The ISR (installed by the OS and in write-protected memory) runs with supervisor privilege. Now O.S. code is running as superman again and can decide what to do.

Note: This is all about privilege. The OS gets supervisor privilege back when the interrupt handler executes. The user process cannot stop this because the user process does not have the authority to block interrupts or to change the handler.

3.6.4 Tool of the week: Make(1)

Make is a program to control program builds automagically, but it can be much, much more.

- Based on dependencies—there’s no need to regenerate a file if its source hasn’t changed.
A dependency looks like: **target: source**.
A particular target can have multiple dependency lines
- Implicit Rules—Make knows how to do certain things (compile C source, for example: if a .o file depends on a .c file if there is one of the same name in the directory).
You can define your own if you want.
- Explicit rules: After a tab, an series of instructions for making the thing:

```
foo.o: foo.c
    gcc -c -Wall -ansi -pedantic foo.c
```

- It supports variables. In particular CC, SHELL, CFLAGS.
\$(VARNAME) to evaluate. \$\$ for a dollar sign
- Remember TABS
(Quite possibly the dumbest decision since **creat()**).
- In rules:

@	do a line silently
#	comment
-	proceed even in the face of errors

- Interfaces nicely with RCS (will check out files for you.)
- To use: **make thing**
if “thing” isn’t specified, it makes the first target.
- Emacs: M-x compile

For more info, read the man page, or the info page.

4 Lecture: The Process Model

Outline:

- Announcements
- From last time
- The Process (Users' View)
- The Operating System's view: The context
- Example of a context switch
- System Calls Again
- System Call Mechanisms
 - How to do it
 - And onwards
- Onwards
- The Process Model: a little deeper
- Pseudoparallelism and nondeterminism
- Possible process states

4.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- Asgn1 due Wednesday (reminder about late days)
- **Make(1)**
- Lab02: Why minix 3.1.8

4.2 From last time

- History
- Make(1) in AM
- System call mechanisms
- Booting
- Regaining control

4.3 The Process (Users' View)

A quick overview of the (human) users' view of the system:

Each running program becomes a process, isolated from all other processes:

- Each process has the illusion of being alone
- Has its own memory
- Has its own scheduling time
- Has its own interface to the outside world (file descriptors)
- All interprocess (and outside world) interaction takes place through the operating system.

Processes have identity:

- User ID
- Group ID
- Process ID

Processes have resources:

- memory (address space)
- time

4.4 The Operating System's view: The context

In the OS's view, a process consists of some resources:

registers	Each process gets its own copy.
address space	A region of memory, usually starting at address 0, corresponding to a particular process.
identity	uid, gid, pid, ppid, etc. All those things that determine a process's identity and privilege.
file descriptors	Connections to the global file descriptor table to allow for IO
signals/masks	Pending notifications

The OS's purpose is to keep these separate. How?

- Records kept in a **process table**
- Processes run for an allotted time(**quantum**), or until they yield (e.g. disk wait)
- **context switch**
 - Old process suspended (timer?)
 - Old process's registers saved (where?)
 - Old process's memory—text,stack,data—saved in a **core image**
 - Memory set up for new process (text,stack,data)
 - Registers set up for new process
 - New process “continued” as if nothing had happened.
Imaging blinking and discovering that it was three hours later.

4.5 Example of a context switch

How does it really happen? (with pictures, and everything.)

- Process A is running (Figure 4.5a)
- An interrupt occurs (Figure 4.5b)
- Push registers to preserve them. (Figure 4.5c)
- Save SP in the process table and switch to the operating system (kernel) stack (Figure 4.5d)
- The OS is now running:
 - Preserve Process A’s memory (if necessary)
 - Choose the next process to run.
 - Load Process B’s memory (if necessary)
- Restore B’s SP from process table. (Figure 4.5e)
- Pop B’s registers (Figure 4.5f)
- Return from interrupt (Figure 4.5g)

4.6 System Calls Again

We said “An OS is defined by its system calls”. What does that mean?

System Call

A system call is the means by which the kernel provides access to a particular operating system service, and the services available through system calls (e.g., reading and writing the disk, allocating memory, starting new processes, etc.) are reserved to the kernel; there is no other way of doing these things.

4.7 System Call Mechanisms

Last time we said

- It’s all about privilege.

4.7.1 How to do it

How done? Machine dependent. usually in assembly, but hidden from the users’ view.

In any case, some sort of trap is involved to get supervisor privileges.

A *trap* is a software generated interrupt. Exactly what happens varies from architecture to architecture, but the result usually involves:

- saving the state of the currently executing program
- elevating of processor privilege level to “supervisor”



Figure 5: The process of a context switch

- transfer of control to a pre-registered routine
- *The OS does something*
- Then, usually:
 - privilege level is reduced back to user
 - original state of executing program is restored

4.7.2 And onwards

- How does the OS get control back?

4.8 Onwards

Last time we talked about context switching and the concept of a Process. Now that we have the general idea, let's look at some general structures.

This week we are going to look at general OS architectures, then move into scheduling.

4.9 The Process Model: a little deeper

Now, we need to talk about some details of how it's actually done and the issues involved.

The **process** is the most important concept in understanding operating systems.

The operating system provides **multiprogramming**:

pseudoparallelism time-sliced parallelism on a uniprocessor

(yield?, pre-empt?)

true parallelism provided by a multiprocessor

Keeping track of parallel activities is difficult, so OS designers have developed the model of the **sequential process**:

Each process (**running program**) has its own:

- IO Access: open file descriptors
- Register file
- Memory
 - Text (code) segment
 - Data (bss and heap) segment
 - Stack segment
- Program Counter

4.10 Pseudoparallelism and nondeterminism

(Ir?)regular (random) context switches mean programs can make no assumptions about:

- order of execution relative to other programs
- timing (example: a clock, or game timing)

(If timing *is* important, you need to get a Real-Time O.S.)

4.11 Possible process states

Not all processes can run at the same time:

At any given time, any process can be in one of three possible states: Running, Ready, or Blocked. Possible transitions between these states are shown in Figure 6.

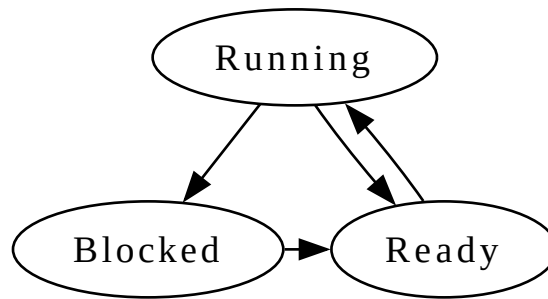


Figure 6: Possible states for a process

In more detail, a process can be:

Running The process is loaded in memory and is currently executing on the cpu.

Ready The process is runnable, but another process has the cpu.

Blocked The process is waiting for some external event to enable it to run. E.g. completion of some IO event, or an alarm, or availability of more memory.

5 Lecture: Operating System Structures

Outline:

- Announcements
- Onwards
- The Process Model: a little deeper
- Pseudoparallelism and nondeterminism
- Possible process states
- Scheduling
- What about IPC?
- Operating System Structures
 - Monolithic Systems
 - Layered Systems
 - Virtual Machines (VM/370, vmware, bochs)
 - Client-server model
- The Layered Architecture of Minix
- Example: Description a MINIX disk interrupt

5.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- `~pnico/longlines.pl`
- why minix 3.1.8
- lab grading scheme
- Lab02 due next wednesday (why not?)
- QEMU says it supports M1

5.2 Onwards

Last time we talked about context switching and the concept of a Process. Now that we have the general idea, let's look at some general structures.

This week we are going to look at general OS architectures, then move into scheduling.

5.3 The Process Model: a little deeper

Now, we need to talk about some details of how it's actually done and the issues involved.

The **process** is the most important concept in understanding operating systems.

The operating system provides **multiprogramming**:

pseudoparallelism time-sliced parallelism on a uniprocessor

(yield?, pre-empt?)

true parallelism provided by a multiprocessor

Keeping track of parallel activities is difficult, so OS designers have developed the model of the **sequential process**:

Each process (**running program**) has its own:

- IO Access: open file descriptors
- Register file
- Memory
 - Text (code) segment
 - Data (bss and heap) segment
 - Stack segment
- Program Counter

5.4 Pseudoparallelism and nondeterminism

(Ir?)regular (random) context switches mean programs can make no assumptions about:

- order of execution relative to other programs
- timing (example: a clock, or game timing)

(If timing *is* important, you need to get a Real-Time O.S.)

5.5 Possible process states

Not all processes can run at the same time:

At any given time, any process can be in one of three possible states: Running, Ready, or Blocked. Possible transitions between these states are shown in Figure 7.

In more detail, a process can be:

Running The process is loaded in memory and is currently executing on the cpu.

Ready The process is runnable, but another process has the cpu.

Blocked The process is waiting for some external event to enable it to run. E.g. completion of some IO event, or an alarm, or availability of more memory.

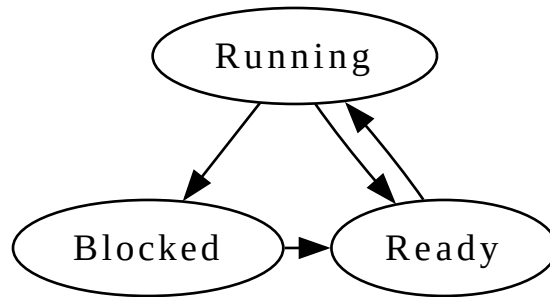


Figure 7: Possible states for a process

5.6 Scheduling

Transitions among the states of Figure 7 are made by the **scheduler**. The scheduler hides the details of starting and stopping processes:

- handles interrupts and dispatches them to the appropriate process
- chooses a process to run from among runnable processes
- Maintains the **process table** that contains
 - program counter
 - stack pointer
 - memory allocation information
 - status
 - open file descriptors
 - other stuff that must be saved (more record keeping: execution times, signal mask, etc.)

Minix divides its process table into separate ones for scheduling, memory, and file information, but the principle is the same.

This leads to the model of the operating system shown in Figure 8.

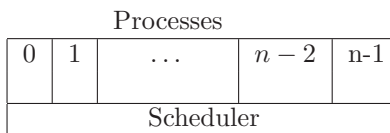


Figure 8: The lowest layer of the operating system

5.7 What about IPC?

Now we have separated processes so they don't know about each other, how do they communicate? IPC must be provided by the O.S.

5.8 Operating System Structures

Now that we've looked at the “external” view of an operating system, how would one be built?

Guts are so inelegant
—Patrick Beard

Operating systems have to do a lot of things:

- Scheduling
- Interprocess Communication
- Memory Management
- Filesystem Management
- Device Management
- Network Management
- *Lots of other things...*

How to put one of these together?

5.8.1 Monolithic Systems

Provide a set of system services through a single interface(a trap handler (Supervisor Call)):

- The system is one big object
- no information hiding
- requires programmer discipline; difficult to maintain
Can be organized well, but be careful...
- Structure:
 - a main() program invoked by the trap
 - a set of service routines that carry out system calls
 - a set of utility routines

Dispatched through a call table.. E.g. `open(name,flags,mode)`

```
movl    $0xFFFFFFFF,%ebx    ; String pointer (filename)
movl    $0x2,%ecx            ; flags (O_RDONLY)
movl    $0x0,%edx            ; mode (0)
movl    $0xb,%eax            ; syscall code for open()
int     $0x80                ; do the call
```

5.8.2 Layered Systems

Generalizes the approach above, separating tasks.

First exhibited in the THE operating system.

Technische Hogdhschool[University] Eindhoven in the Netherlands, by Dijkstra in 1968

The machine: Electrologica X8, with 32K of 27-bit words (400kHz).

The OS Six layers:

Layer	Function
5	The Operator
4	User Programs
3	IO Management (abstract IO devices)
2	Operator-process communication (individual operator consoles)
1	Memory and drum (512K words) management
0	Processor allocation and multiprogramming

Level 3 is above level 2 because the devices might need to speak to the operator in case of malfunction.

Multics did this with concentric rings.

How did THE enforce these layers? It didn't, but it was a good design tool.

5.8.3 Virtual Machines

(VM/370, vmware, bochs)

Go all the way: If we're going to virtualize this machine, let's do it:

- VM/370: Virtual Machine Monitor
- Different users can run different OSs
 - CMS: Conversational Monitor System (interactive)
 - OS/360: batch processing
 - vm/370 in itself...
- Used by windows to run MS-DOS programs. (full emulation vs. partial emulation)
- VMare does the same thing completely (as do bochs and qemu)

5.8.4 Client-server model

Mach/Minix

Pull it all out of the kernel. (Well, not *all*)

servers and clients communicate via message passing.

The (now) *micro*-kernel does only things really requiring privilege and delegates to user-level processes.

- Separates **mechanism** from **policy**
- message-passing
- modifiable
- could be remote(?)

e.g.: Mach

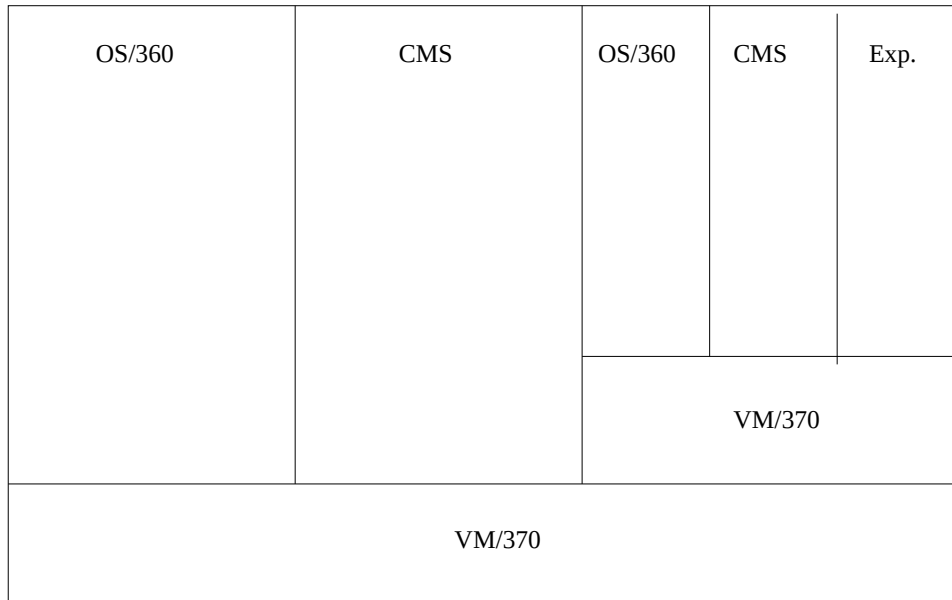


Figure 9: Structure of VM/370

5.9 The Layered Architecture of Minix

The structure of minix is as in Figure 10.

Layer 1 Sheduling, handling traps and interrupts, facilitating message passing.

The bottom is written in assembly(must be), the rest is C.

Layer 2 IO Processes, one per device type. Compiled into the kernel, but have separate identities, (and, if supported, privilege levels)

Layer 3 Server processes: File System, Memory Manager, Network Server, etc. These are user-level processes that implement the system calls.

These run with higher priority than user processes and never terminate while the system is running.

Layer 4 User processes

5.10 Example: Description a MINIX disk interrupt

- Structure of MINIX
- How a disk read happens in MINIX

User Level	Init	cat	...			
System Servers	filesystem		memory manager	info.	reincarnation	...
Device Tasks	Disk	ethernet	memory	tty	...	
Kernel	(micro)Kernel				Clock	System

Figure 10: Layout of the Minix system

6 Lecture: Intro. to Concurrency

Outline:

- Announcements
- Once again: Processes
- Lightweight Processes: Threads
- Aside: Reentrancy
- Introduction to Asgn2
 - Nine little functions
 - *Scheduling
 - Demonstration: threading in action
 - Review: context switch
 - A thread's context: stack and registers
 - *Stack structure: The C calling convention
- How to do it
 - Race condition, defined
- Critical Sections
- Mutual Exclusion
- Busy Waiting: Software Only
- From last time: Review of Busy Waiting
- Peterson's Solution

6.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- In spite of saying “no support”, I posted instructions for building individual tests on the forum
- `~pn-cs453/demos/tryAsgn1 2>&1 | tee logfile`
- MEANINGFUL SYMBOLS (not: SIXTEEN)
- gradebook
- `mkfs.mfs` on 3.1.8 (Read the durned footnote)

6.2 Once again: Processes

Recall (those of you who were awake) the process of a context switch:

- Old process suspended (timer?)
- Old process's registers saved (where?)
- Old process's memory—text,stack,data—saved in a **core image**
- Memory set up for new process (text,stack,data)
- Registers set up for new process
- New process “continued” as if nothing had happened.

Imaging blinking and discovering that it was three hours later.

6.3 Lightweight Processes: Threads

Traditional Operating Systems only support processes as above.

Sometimes we don't need that kind of isolation.

Other times we don't **want** that kind of isolation.

Sometimes it's necessary (or at least desirable) to have multiple independent threads of control in a single address space: filesystem cache, web browser, etc.

Types of threads:

user-level all at the user level (advantages/disadvantages)

POSIX P-threads, Mach C-Threads.

- lightweight (very)
- requires no kernel support. (if a thread blocks, the whole process blocks.)
- has no kernel support. (relies on good behavior—the reason we have a kernel in the first place)

kernel-level With kernel support level (advantages/disadvantages)

- More robust
- More expensive (kernel must be aware of threads to schedule them, resulting in a “real” context switch.)

Unfortunately, threads changes the programming model:

- Break the model of **sequential process**. no more illusion of sequentiality.
- Hard to retrofit: hidden assumptions (jet engine on a car?)
- Semantic questions:
 - What about **errno**?
 - Signal delivery? alarms?
 - fork(): does the child have all running threads?

- fork(): what about blocked threads? (couldn't happen w/o threads)
- Stack management: how to detect stack overflow and fix it.
- issues.
 - non-preemptive
 - preemptive

6.4 Aside: Reentrancy

Reentrant code is code that written such that a single copy in memory can be shared between many applications at once. That is, a reentrant function is one where it is possible to safely have more than one activation at the same time.

What that means:

- no self-modifying code
- no static variables (except those that actually pertain to truly global state).

Examples:

`strcat()` is reentrant.
`strtok()` is not.

6.5 Introduction to Asgn2

6.5.1 Nine little functions

<code>lwp_create(function, argument)</code>	create a new LWP
<code>lwp_start(void)</code>	start the LWP system
<code>lwp_yield(void)</code>	yield the CPU to another LWP
<code>lwp_exit(int)</code>	terminate the calling LWP
<code>lwp_wait(int *)</code>	wait for a thread to terminate
<code>lwp_gettid(void)</code>	return thread ID of the calling LWP
<code>tid2thread(tid)</code>	map a thread ID to a context
<code>lwp_set_scheduler(scheduler)</code>	install a new scheduling function
<code>lwp_get_scheduler(void)</code>	find out what the current scheduler is

Scheduling

The lwp scheduler is a structure that holds pointers to five functions. These are:

void init(void) This is to be called before any threads are admitted to the scheduler. It's to allow the scheduler to set up. This one is allowed, to be NULL, so don't call it if it is.

void shutdown(void) This is to be called when the lwp library is done with a scheduler to allow it to clean up. This, too, is allowed, to be NULL, so don't call it if it is.

void admit(thread new) Add the passed context to the scheduler's scheduling pool.

void remove(thread victim) Remove the passed context from the scheduler's scheduling pool and from the global thread list.

thread next(void) Return the next thread to be run or NO.THREAD if there isn't one.

Changing schedulers will involve initializing the new one, pulling out all the threads from the old one (using `next()` and `remove()`) and admitting them to the new one (with `admit()`), then shutting down the old scheduler.

6.5.2 Demonstration: threading in action

- `nums`
- `randomsnakes`
- `hungrysnakes`
- `nums` with different scheduling algorithms
 - `z` choose the process in slot 0
- `hungrysnakes` with different scheduling algorithms
 - `z` choose the process in slot 0
 - `h` choose the process that has eaten the most
 - `l` choose the process that has eaten the least

6.5.3 Review: context switch

- Old process suspended (timer?)
- Old process's registers saved (where?)
- Old process's memory—text, stack, data—saved in a **core image**
- Memory set up for new process (text,stack,data)
- Registers set up for new process
- New process “continued” as if nothing had happened.

6.5.4 A thread's context: stack and registers

Registers The register structure of the 64-bit Intel x86_64 is shown in Table 1.

<code>rax</code>	General Purpose A	<code>r8</code>	General Purpose 8
<code>rbx</code>	General Purpose B	<code>r9</code>	General Purpose 9
<code>rcx</code>	General Purpose C	<code>r10</code>	General Purpose 10
<code>rdx</code>	General Purpose D	<code>r11</code>	General Purpose 11
<code>rsi</code>	Source Index	<code>r12</code>	General Purpose 12
<code>rdi</code>	Destination Index	<code>r13</code>	General Purpose 13
<code>rbp</code>	Base Pointer	<code>r14</code>	General Purpose 14
<code>rsp</code>	Stack Pointer	<code>r15</code>	General Purpose 15

Table 1: Integer registers of the x86 CPU

Stack structure: The C calling convention

The extra registers available to the x86_64 allow it to pass some parameters in registers. This makes the overall calling convention a little more complicated, but, in practice, it will be easier for your program since you won't be passing enough parameters to push you out of the registers onto the stack.

The steps of the convention are as follows (illustrated in Figures 11a–f):

- a. **Before the call** Caller places the first six integer arguments into registers `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, and `%r9`. If there are more, they are pushed onto the stack in reverse order. This is shown in the figure, but you won't encounter more in this assignment.

- b. **After the call** The `call` instruction has pushed the return address onto the stack.

- c. **Before the function body** If the function has more parameters and local variables than will fit into the registers it will execute the following two instructions to set up its frame:

```
pushq %rbp
movq %rsp,%rbp
```

Then, it adjusts the stack pointer to leave room for any locals it may need.

Your functions probably will not need to set up a frame.

- d. **Before the return** If the function has set up a call frame in the previous step it needs to clean up after itself. To do this, before returning it executes a `leave` instruction. This instruction is equivalent to:

```
movq %rbp,%rsp
popq %rbp
```

The effect is to rewind the stack back to its state right after the call.

If the function did not set up a frame, its `%rsp` register is still pointing to the return address.

- e. **After the return** After the return, the Return address has been popped off the stack, leaving it looking just like it did before the call.

Remember, the `ret` instruction, while called “return”, really means “pop the top of the stack into the program counter.”

- f. **After the cleanup** Finally, the caller pops off any parameters on the stack and leaves the stack is just like it was before.

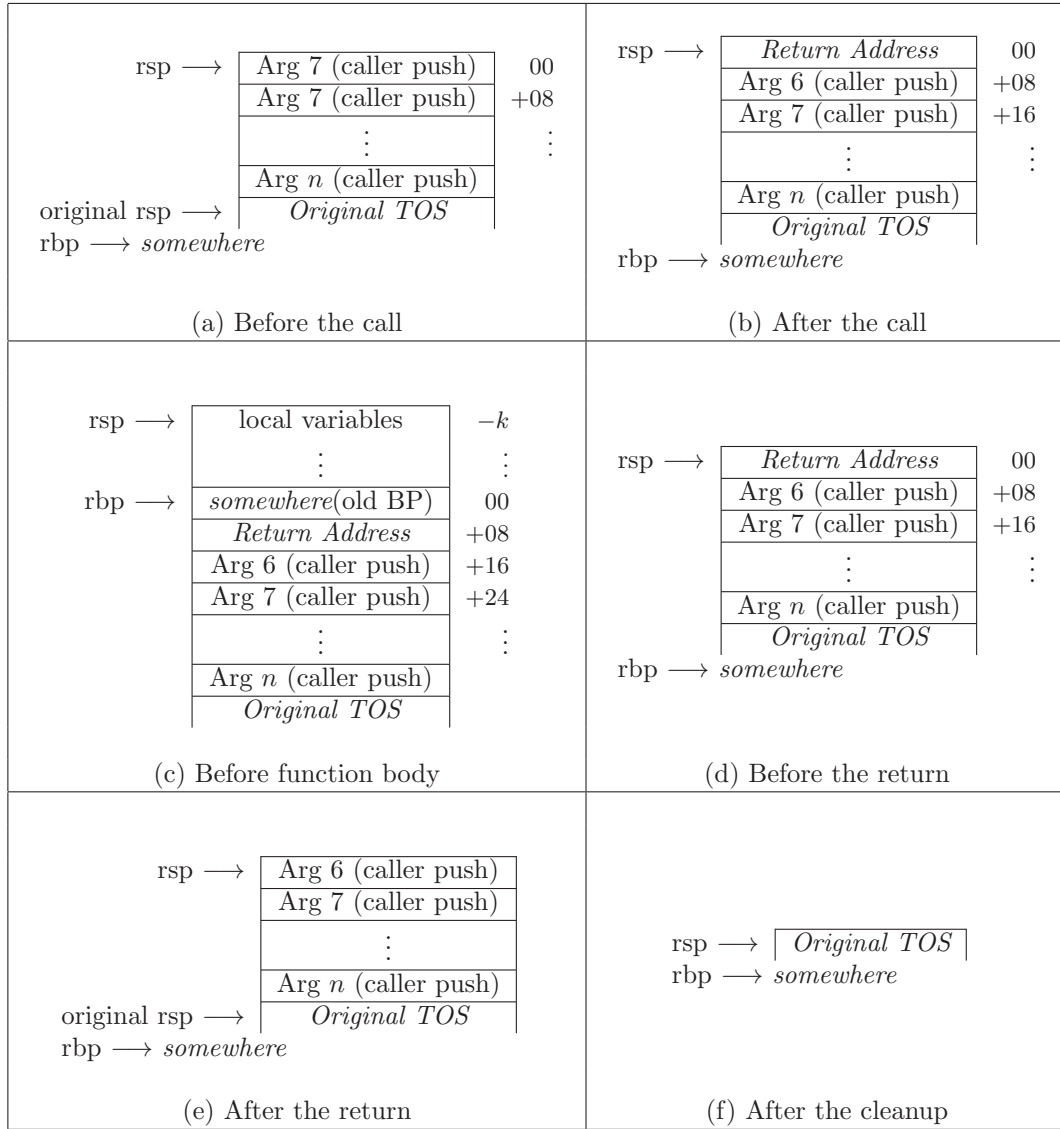


Figure 11: Stack development (Remember that the real stack is upside-down)

Example (real gcc-generated code):

dummy:	pushq %rbp movq %rsp, %rbp movq %rdi, -24(%rbp) movq -24(%rbp), %rax movq %rax, -8(%rbp) leave ret	long dummy(long x) { long tmp; tmp = x; return tmp; }
main:	pushq %rbp # set up frame movq %rsp, %rbp → movl \$5, %edi # load argument call dummy # do call leaveq retq	int main(){ dummy(5); }

	(dummy) x:	-24
	(dummy)	-16
	(dummy) tmp:	-08
dummy's rbp →	(dummy) saved rbp (main())	00
	(dummy) Return Address	+08
	(main) saved rbp (start())	+16
	(main) return address	+24

The context type:

```
typedef struct threadinfo_st *thread;
typedef struct threadinfo_st {
    tid_t      tid;           /* lightweight process id */
    unsigned long *stack;     /* Base of allocated stack */
    size_t      stacksize;    /* Size of allocated stack */
    rfile       state;        /* saved registers */
    thread       lib_one;      /* Two pointers reserved */
    thread       lib_two;      /* for use by the library */
    thread       sched_one;    /* Two more for */
    thread       sched_two;    /* schedulers to use */
} context;
```

The scheduler type:

```
/* Tuple that describes a scheduler */
typedef struct scheduler {
    void (*init)(void);       /* initialize any structures */
    void (*shutdown)(void);    /* tear down any structures */
    void (*admit)(thread new); /* add a thread to the pool */
    void (*remove)(thread victim); /* remove a thread from the pool */
    thread (*next)();          /* select a thread to schedule */
} *scheduler;
```

The lwp functions:

<code>lwp_create(function,argument,stacksize)</code>	create a new LWP
<code>lwp_gettid()</code>	return thread ID of the calling LWP
<code>lwp_exit()</code>	terminates the calling LWP
<code>lwp_yield()</code>	yield the CPU to another LWP
<code>lwp_start()</code>	start the LWP system
<code>lwp_stop()</code>	stop the LWP system
<code>lwp_set_scheduler(scheduler)</code>	install a new scheduling function
<code>lwp_get_scheduler(scheduler)</code>	find out what the current scheduler is
<code>tid2thread(tid)</code>	map a thread ID to a context

Tools for stack manipulation:

Macros for stack manipulation: Remember, these **must** be called first and last.

<code>GetSP(var)</code>	Sets the given variable to the current value of the stack pointer.
<code>SetSP(var)</code>	Sets the stack pointer to the current value of the given variable.

`void swap_rfiles(rfile *old, rfile *new).` This does two things:

1. if `old != NULL` it saves the current values of all 16 registers to the `struct registers` pointed to by `old`.
2. if `new != NULL` it loads the 16 register values contained in the `struct registers` pointed to by `new` into the registers.

General Notes:

- Remember to allocate enough space for your stacks
- Remember stacks are **upside down**
- Draw pictures!
- Don't even think of turning the optimizer on!
- "Remember" `gcc -S`
- "remember" function pointers

```
typedef void (*lwpfun)(void *); /* type for lwp function */
```

- be careful of locals when climbing from one stack to another
- what happens if your function returns? (Build a wrapper function?)

6.6 How to do it

All (most of?) the magic is in creating the thread:

- Allocate the stack
- Figure out what you want the world to look like when the thread starts running

- Wind up the stack so that when `yield()` chooses the thread, loads its context with `swap_rfiles()` and **returns to it** that the world looks like that.
- The endgame of `swap_rfiles()` is:

```
leave
    movq %rbp, %rsp
    popq %rbp
ret
    popq %rip
```

7 Lecture: Concurrency and Synchronization

Outline:

Announcements
Problems with parallelism: Race Conditions
 Race condition, defined
Critical Sections
Mutual Exclusion
Busy Waiting: Software Only
From last time: Review of Busy Waiting
Peterson's Solution

7.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- Warning about stack overflow/etc. E.g. WNOHANG or WUNTRACED. RTFM
- See gradesheet snapshot
- Lab02:
 - Choosing an emulator: Pick one that works for you. (Pick VirtualBox)
 - Be sure virtualization is turned on in your BIOS.
- “remember” function pointers

```
typedef void (*lwpfun)(void *); /* type for lwp function */
```

- Partners: Only turn in one assignment/lab with both names.
- GDB hints: display, conditional breakpoints

7.2 Problems with parallelism: Race Conditions

As soon as there any shared resources (even the filesystem), there can be problems.

Example: Print spooler

Consider a print spooler where files to be printed are placed in spots in an array indexed by `qhead`. The process to add an element looks something like: `array[qhead++]=name`. This breaks into three stages: read, update, write.

Consider the effect of the following interleaving.

Process A	Process B
(1) Read <code>qhead</code>	(2) Read <code>qhead</code>
	(3) Update array
(4) Update array	(5) Increment Index
(6) Increment index	
(7) Write <code>qhead</code>	(8) Write <code>qhead</code>

Not so good.

7.2.1 Race condition, defined

A race condition is any situation where the precise ordering of a series events affects the (correctness of the) outcome of the entire process.

The term is usually only applied where processes are reading or writing some shared data and where correctness is at stake.

Race conditions manifest themselves as nondeterministic behavior (usually leading to strange behavior at inopportune times).

7.3 Critical Sections

Critical sections require mutual exclusion.

For good solution we wish to maintain the following four principles:

1. No two processes may simultaneously enter the critical region.
2. No assumptions may be made about CPU speed or the number of CPUs
3. No process running outside of its critical section may run while another process is in its critical section.
4. No process should have to wait forever in its critical section.

7.4 Mutual Exclusion

How can we ensure mutual exclusion?

7.5 Busy Waiting: Software Only

For busywaiting to work, all contention must be among time-sliced processes or with true concurrency. (Why)

These solutions do not rely on hardware support to work correctly.

Options:

- Eliminate concurrency: That'll fix it.
- Hope for the best?

- Disabling Interrupts

Advantages: Foolproof?

Disadvantages: Gives user (fool?) too much power. Blocks everything. (Decapitation will cure a headache.)
What about a multiprocessor?

This is used in the kernel.

- Lock Variables (Software Only)

```
while ( TRUE ) {
    while ( lock )
        /* twiddle */;
    lock = 1;
    critical_things();
    lock = 0;
    noncritical_things();
}
```

Advantages: Allows finer-grained synchronization

Disadvantages: **Doesn't work** (race-conditions)

- Strict Alternation: use a variable to say whose turn it is, have each thread set it explicitly on the way out

Advantages: Works.

Disadvantages: Significantly reduces parallelism. What if a process forgets? What if a process is slow?

Process A

```
while ( TRUE ) {
    while ( turn != 0 )
        /* twiddle */;
    critical_things();
    turn = 1;
    noncritical_things();
}
```

Process B

```
while ( TRUE ) {
    while ( turn != 1 )
        /* twiddle */;
    critical_things();
    turn = 0;
    noncritical_things();
}
```

7.6 Peterson's Solution

A better solution: Peterson's Solution.

The first safe software solution was proposed by Dekker and published by Dijkstra in 1965 ("Co-operating Sequential Processes," in *Programming Languages*, London: Academic Press, 1965.)

Peterson came up with a better solution in 1981.

See Figure 12.

Advantages: Works. Does not require strict alternation.

Disadvantages: Busy waiting.

```

#define TRUE 1
#define FALSE 0
#define N 2           /* number of processes */

int turn;              /* whose turn is it? */
int interested[N];     /* all values initially FALSE */

void enter_region(int self) { /* self is 0 or 1 */
    int other;             /* number of the other process */

    other = 1-self;

    interested[self] = TRUE; /* show interest */
    turn = self;             /* try and claim the turn */
    while ((turn==self) && (interested[other]==TRUE))
        /* dum-dee-dum */;
}

void leave_region(int self) { /* process who is leaving */
    interested[self]=FALSE;
}

```

Figure 12: Peterson's solution for mutual exclusion

8 Lecture: Concurrency and Synchronization, cont.

Outline:

Announcements
LWP
From last time: Review of Busy Waiting (Software only)
Onwards: Busy Waiting: With Hardware Support
Reflection
 sleep() and **wakeup()**
Synchronization without busy waiting
Semaphores
 Monitors: (Hoare 1974, Brinch Hansen 1975)
More Interprocess Communication

8.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- “remember” function pointers

```
typedef void (*lwpfun)(void *); /* type for lwp function */
```

- Lab02

- `testfloppy.img` is outside of minix, in the host filesystem. Inside, it's `/dev/fd0`.
- Don't just click the little 'X' to stop minix

8.2 LWP

- The scheduler is totally separable. You can test it with `libPLN.a`.
- My demo code is in `~pn-cs453/Given/Asgn2`. It's not a particularly good test, but it's fun, `~pn-cs453/demos/tryAsgn2` will give it more of a workout.
- There are four pointers in the `context` structure. Two are reserved for the scheduler, two for the library. Use them for anything of nothing.

Consider the value of:

```
#define sched_one next
```

- The thread you create in `lwp_create()` will be loaded when one of `lwp_yield()`, `lwp_start()`, or `lwp_exit()` chooses it and passes its context to `swap_rfiles()`.
- It is loaded in the middle of `swap_rfiles()`, so it will go through `swap_rfiles()`'s `leave;` `ret` endgame. Figure out where you want to wind up, and work your way backwards through those instructions to see what you want your context to look like when it's loaded.
- the secret of `lwp_exit()`.
- Remember stacks are upside-down.
- Remember: all zeros is not a valid floating point state. Use the initializer.

8.3 From last time: Review of Busy Waiting (Software only)

Methods discussed:

- Hope for the best
- Disabling Interrupts
- Lock Variables (Software Only)
- Strict Alternation
- Peterson's Solution

The problem: lack of atomicity.

8.4 Onwards: Busy Waiting: With Hardware Support

Peterson's soln is fairly complicated. What if we can have a little help.

- Test-and-set-lock instruction(TSL): Read a memory location and set a non-zero value to it.

Advantages: Atomic access guarantees simple correctness.

Disadvantages: Requires hardware support, still busy waiting

enter: tsl r1,lock	leave: move lock,#0
cmp r1,#0	ret
jne enter	
ret	

8.5 Reflection

The fundamental busywaiting problem: Wasting time.

That is, *the waiting process can actively prevent the event for which it is waiting.* (feeding the cat?) Remember the four desired principles:

1. No two processes may simultaneously enter the critical region.
2. No assumptions may be made about CPU speed of the number of CPUs
3. No process running outside of its critical section may run while another process is in its critical section.

4. No process should have to wait forever in its critical section.

Reconsider the problems with busywaiting:

- Except on a multiprocessor, busywaiting *must* waste time.
- Consider the priority inversion problem: a low-priority process is holding a lock that a high-priority process needs, but the low priority process can't run while the high-priority one is waiting.

8.5.1 sleep() and wakeup()

sleep() goes to sleep until awakened
wakeup(process) wakes up a sleeping process

<pre> #define N 100; int count=0; void producer(void) { while(TRUE) { produce(); if (count == N) /* full */ sleep(); enter_item(); count = count + 1; if (count == 1) /* empty */ wakeup (consumer); } } </pre>	<pre> void consumer(void) { while(TRUE) { if (count == 0) /* empty */ sleep(); remove_item(); count = count - 1; if (count == N-1) /* full */ wakeup (producer); consume(); } } </pre>
---	--

Figure 13: A producer-consumer implementation with a race condition

Race conditions still: Wakeup might be missed:

When the buffer is empty:

1. Scheduler interrupt **consumer()** after the **count==0** test, but before the **sleep()**.
2. **producer()** produces an item, notes that count is now 1, and **wake()**s **consumer()**
3. the **wake()** is lost because **consumer()** is not asleep.
4. **consumer()** is scheduled, then goes to sleep.
5. **producer()** continues to produce until the buffer is full, then goes to sleep.

Nobody ever wakes up.

8.6 Synchronization without busy waiting

8.7 Semaphores

Generalized `sleep()` and `wakeup()` using a counter.

```
p(counter)  down(counter)  decrement counter; wait if zero
v(counter)  up(counter)    increment counter; wakeup others if was zero
```

Must be **atomic**:

- usually done as a system call with disabled interrupts—note this is not the same as a long-term busywait.
- LWP can turn off signals to achieve the same effect.

Semaphores can be used for both (And these are **different**)

- mutual exclusion (binary semaphore)
- synchronization (initially N for producer-consumer)

Figure 14 shows a solution to the producer-consumer problem using semaphores.

Still must be careful: **if you lock in the wrong order, you can deadlock.** consider the effect of reversing the downs in figure 14.

<pre>#define N 100 semaphore mutex = 1; semaphore empty = N; semaphore full = 0; void producer(void) { while(TRUE) { produce(); down(&empty); down(&mutex); enter_item(); up(&mutex); up(&full); } }</pre>	<pre>void consumer(void) { while(TRUE) { down(&full); down(&mutex); remove_item(); up(&mutex); up(&empty); consume(); } }</pre>
---	---

Figure 14: A semaphore-based producer-consumer implementation

8.7.1 Monitors: (Hoare 1974, Brinch Hansen 1975)

Recall the sensitivity of semaphores to ordering we saw in the example of the other time: If the producer and consumer lock `mutex` and `full/empty` in the wrong order, they will deadlock.

Monitors are a higher-level synchronization mechanism requiring programming-language support.

- A monitor is region of code where only one process can be active at a time. (enforced by the language).

- Communication between processes is done via **condition variables** with the primitives:

wait(condition) wait until a signal. A process that blocks in the monitor releases other processes to enter.

Note that this will be ok, because the release is *voluntary*.

signal(condition) send a signal. To enforce the exclusion principle, a process that signals is required to leave the monitor immediately.

A signal wakes one process waiting on that condition variable.

These are very like sleep and wakeup, but automagically synchronized so counters are unnecessary.

A monitor-based solution to the producer-consumer problem is shown in Figure 15.

8.8 More Interprocess Communication

Ok, so, looking at our mechanisms that work

Spin Locks:	Waste Time
Semaphores:	Complicated (easy to get wrong)
Monitors:	Require language support

So what else can we do?

<pre> monitor ProducerConsumer condition full,empty; integer count; procedure enter() begin if count = N then wait(full); enter_item(); count := count + 1; if count = 1 then signal(empty); end; procedure remove() begin if count = 0 then wait(empty); remove_item(); count := count - 1; if count = N-1 then signal(full); end; count := 0; end monitor </pre>	
<pre> procedure consumer() begin while TRUE do begin ProducerConsumer.remove(); consume_item(); end end; end; </pre>	<pre> procedure producer() begin while TRUE do begin produce_item(); ProducerConsumer.enter(); end end; end; </pre>

Figure 15: A monitor-based solution to the producer-consumer problem.

9 Lecture: More Synchronization

Outline:

Announcements
LWP?
More Interprocess Communication
 Further generalization: message passing
Robust Programming Clinic
Classic IPC Problems

9.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- Don't just click the little 'X' to stop minix
- function pointers
- schedulers
- context structure
- tryAsgn2

9.2 LWP?

- Assignment news:
 - Yes, really.
- Winding up the stack
 1. Figure out what you want the world to look there (executing the first instruction of threadfun with the universe looking like it'd just been called)
 2. Figure out how you're going to get there. `swap_rfiles()`'s endgame:

```
<load context>
leave
ret
```

Leave is:

```
movq %rbp, %rsp
```

```
popq %rbp
```

So, putting it all together, the endgame is:

```
movq %rbp, %rsp ; copy base pointer to stack pointer
```

```
popq %rbp      ; pop the stack into the base pointer
```

```
ret           ; pop the stack into the instruction pointer
```

3. work backwards through this to figure out what you want the loaded context to look like so you wind up where you wanted to be in step 1.

You know what you need to know from the description of the normal linkage, even though you're doing something distinctly abnormal.

- Function pointers
- Don't separate `{load,store}_state()`.
- `tryAsgn2`

9.3 More Interprocess Communication

Ok, so, looking at our mechanisms that work

Spin Locks: Waste Time

Semaphores: Complicated (easy to get wrong)

Monitors: Require language support

So what else can we do?

9.3.1 Further generalization: message passing

If you need me, just whistle.

Monitors require language support, a tricky thing in the “real world”. Message passing gets much of the same safety using operating system support.

Processes communicate via two primitive functions:

- `send(dst,message)`
- `receive(src,message)`

Where do they get sent? (How do we address them?)

Options:

- individual processes
- mailboxes (possibly multiple consumers)

In either case, there may or may not be buffers. If there is a buffer, the OS has to manage this buffer (a meta-producer-consumer problem :). If not, a sender is stopped until there is a receiver: rendezvous

Issues with message passing.

- messages can be lost (acknowledgement)
- messages can be duplicated (sequence numbers)

- messages can be forged (authentication)
- performance:
 - Copying overhead (copy, copy, copy)
 - Transmission latency (You may as well do it yourself)

But it can elegantly solve some problems, as with the producer-consumer solution given in Figure 16. Message passing producer-consumer: send empty messages and send back full ones.

<pre> #define N 100 void producer(void) { message m; thing item; while(TRUE) { produce(&item); receive(consumer,&m); build_message(&m,item) send(consumer,&m); } } </pre>	<pre> void consumer(void) { message m; thing item; int i; /* send N empties */ for(i=0;i<N;i++) send(producer,&m); /* go to work */ while(TRUE) { receive(producer,&m); extract_item(&m,item); send(producer,&m); consume(&item); } } </pre>
---	---

Figure 16: A message-passing solution to the producer-consumer problem.

9.4 Classic IPC Problems

- Producer/Consumer (been there, done that?)
- Dining Philosophers(1965): synchronization rite of passage.
Competing access for limited resources.
Issues:
 - deadlock
 - starvation
- Readers/Writers: e.g. a database.
Issues:
 - Many readers can work simultaneously, but only one writer.

- Starvation: If we let all the readers by (no problem), the writer may starve.
- perhaps an aging scheme?

10 Lecture: Scheduling

Outline:

- Announcements
- We talked a lot about LWP
- So what: Scheduling
 - Process States
 - Policy vs. Mechanism
 - Process types
 - When to schedule
 - Evaluation Criteria
 - Non-preemptive scheduling: run-to-completion
 - Preemptive scheduling

10.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- Don't just click the little 'X' to stop minix
- tryAsgn2
- Call stacks, Algol 60
- Wind up a stack.
- Do atomic swaps

10.2 We talked a lot about LWP

List of stuff to talk about wrt asgn02:

- Draw stack
- pointer arithmetic in size of pointee
- Allocate enough stack
- Do atomic swaps (w/`swap_rfiles()`)
- Be paranoid
- **Alignment** — it's a thing

10.3 So what: Scheduling

We talked a lot about running along until we block, then pick who gets to run, but how is this done?

10.3.1 Process States

Remember at any given time, any process can be in one of three possible states: Running, Ready, or Blocked. Possible transitions between these states are shown in Figure 17.

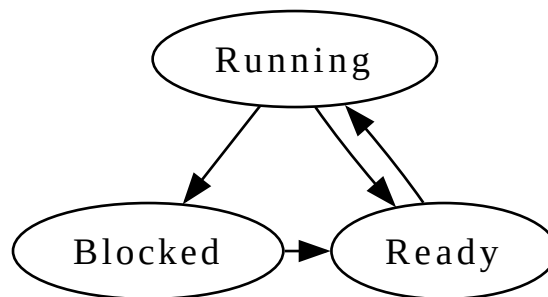


Figure 17: Possible states for a process

10.3.2 Policy vs. Mechanism

policy How we want things to behave. (graduate students should finish within 7 years.)

mechanism How we're going to make it happen. (cut off funding. (vs. throwing them out.))

We're pretty clear on how it happens (particularly after asgn2), but what is it we want to do anyway?

10.3.3 Process types

Processes are usually roughly categorized into one of two different types

IO Bound characterized by short bursts of computation before blocking on IO (or a semaphore)

- Might want to give priority because they can get done and go back to sleep. (hide IO latency)
- Also, more likely to be interactive.

Compute Bound characterized by long bursts of computation before blocking on IO (or a semaphore)

These are dynamic. A process may move back and forth.

10.3.4 When to schedule

Scheduling is mandatory in two cases:

1. When a process exits

2. When a process blocks

It might be desirable under a few other conditions:

1. When a new process is created
(Consider the situation of parent and child after fork()ing)
2. When an IO interrupt occurs
3. When a timer interrupt occurs

10.3.5 Evaluation Criteria

What makes a good algorithm?

Fairness Make sure each process gets its fair share

Efficiency/Utilization keep the CPU busy 100 percent of the time

Response Time minimize response time for **interactive** users

Turnaround minimize turnaround time for **batch** users

Throughput maximize the number of jobs processed per time.

Faster, better, cheaper, choose two.

10.3.6 Non-preemptive scheduling: run-to-completion

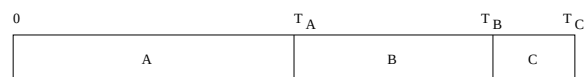
- Run to completion/blockage

Examples:

1. FCFS
2. Shortest Job First:
 - Provably optimal
 - Problem: Starvation and how do you know?

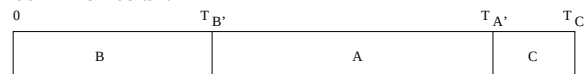
Add an aging function?

The proof:



$$T_{\text{turnaround}} = \frac{T_A + T_B + T_C}{3}$$

Now place the shorter *B* ahead of *A*:



$$\begin{aligned}
T'_{\text{turnaround}} &= \frac{T'_B + T'_A + T_c}{3} \\
&= \frac{T'_B + T_B + T_c}{3}
\end{aligned}$$

But we know that $T'_B < T_A$, so

$$T'_{\text{turnaround}} < T_{\text{turnaround}}$$

We have invented bubble sort.

10.3.7 Preemptive scheduling

Round Robin Every (runnable) process goes in turn.

11 Lecture: More Scheduling

Outline:

- Announcements
 - Preemptive scheduling
- Minix architecture revisited
- Minix 2 Scheduling
- Minix 3 Scheduling
 - Three-level scheduling
- Scheduling Example
- Minix IPC
- Minix structures
- Where do we go from here?

11.1 Announcements

- Old exams on the web site (warning...)
- Coming attractions:

Event	Subject	Due Date		Notes
asgn3	dine	Fri	May 1	23:59
lab03	problem set	Mon	May 4	23:59
midterm	stuff	Wed	May 6	
lab04	scavenger hunt II	Wed	May 13	23:59
asgn4	/dev/secret	Fri	May 22	23:59
lab05	problem set	Mon	Jun 1	23:59
asgn5	minget and minls	Wed	Jun 3	23:59
asgn6	Yes, really	Fri	Jun 5	23:59

Use your own discretion with respect to timing/due dates.

11.1.1 Preemptive scheduling

Priority Sheduling Give each job (or class of jobs) a priority (rank, price, etc) and choose the most important one.

Idea: Break priorities into classes: IO Bound first, then compute-bound. Why?

Example: CTSS (Compatible Time Sharing System):

- large quantum for CPU-bound jobs:
- Processes that use the whole quantum move down.
- Processes doing IO move up to the top again.

Other possibilities for granting priority:

- Process already in memory?
- Locks held/needed
- Resource requirements
- Shortest remaining work next (past performance might be a good predictor of future results?)

Guaranteed/Lottery Give each process its due.

Real Time A whole other course

11.2 Minix architecture revisited

The structure of minix is as in Figure 18.

4	init	proc A	proc B	proc C	...		User Processes
3	File Sys.	Network	Info. Srv.	Proc. Mgr.	Reincarnation Srv.	...	Server Processes
2	Disk	Tty	Ethernet	Memory	...		I/O Tasks
1	scheduling, interrupts, communication				Clock Task	System Task	Process Management

Figure 18: Layout of the Minix system

Layer 1 Sheduling, handling traps and interrupts, facilitating message passing.

The bottom is written in assembly(must be), the rest is C.

Clock and System Tasks exist here.

Layer 2 IO Processes, one per device type. Compiled into the kernel, but have separate identities, (and, if supported, privilege levels)

Layer 3 Server processes: File System, Memory Manager, Network Server, etc. These are user-level processes that implement the system calls.

These run with higher priority than user processes and never terminate while the system is running.

Layer 4 User-level processes.

11.3 Minix 2 Scheduling

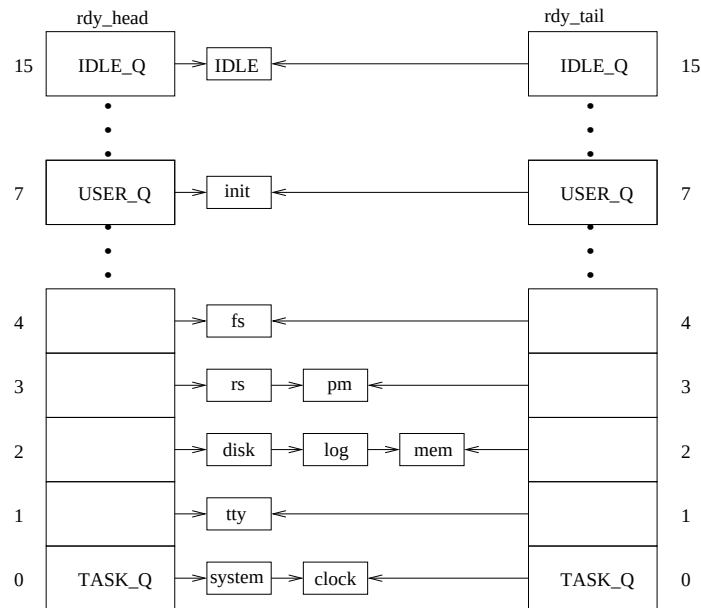
Three-level priority scheduling system:

Level 1	immediate (so no queue)
Level 2	run to completion/block
Level 3	run to completion/block
Level 4	Round-robin. 100ms quantum

Remember the idle process

11.4 Minix 3 Scheduling

Sixteen-level priority scheduling system:



- Variable quantum size set in the process table
- A process that uses its entire quantum will be given another one and sent to the end of the queue.
- Potential change of priority level for processes using complete quanta:
 - If it uses its whole quantum and is its own successor: drops a level.
 - If it was not its own predecessor, rises a level, capped in process table.
- Processes that block with time remaining in their quantum are moved to the head of the queue when they return, but only with the remaining part of their quanta.

11.4.1 Three-level scheduling

For *any* scheduling scheme, it might be worthwhile to treat processes in memory and processes on disk differently.

Processes can be scheduled at three different locations:

admission The system can decide whether or not to admit new jobs, and what to do with them.

memory Constraints are different for jobs in memory and on disk

disk (above)

11.5 Scheduling Example

Perhaps?

11.6 Minix IPC

Three primitives of Minix IPC:

Synchronous

```
send(dest, &message);  
receive(source, &message);  
send_rec(dest_src, &message);
```

Asynchronous

```
notify(dest);
```

Messages (other than `notify()`) communicated via **rendezvous** semantics: The sender waits until the receiver gets the message. Notify is asynchronous.

Why?

- simplifies buffer management.
- more predictable: there is no question of a program behaving differently given a different buffer size (out of its control)

(How would one find out the buffer size?)

11.7 Minix structures

To prepare for the next part of the course, read Sections 2.5–2.6 (this time for real):

- Understand the basic structure of the OS
- Be careful and read critically: There is much to learn from observing OS code. There are also places where OS writers do things that ordinary C programmers should not. (“advanced strategy” — G. Fink)

Remember: design considerations for an OS are different from “mortal programs”. (e.g. consistency above all else `panic()`)

- Read the code; you will eventually anyhow.

11.8 Where do we go from here?

Three major operational areas for any operating system: (specializations of magic?)

IO How does information get in and out of the system and get where it’s going.

Memory Management Making sure that things are where programs expect them to be.

Filesystem Making sure things stay where put and that everything operates efficiently.

12 Lecture: MINIX IO Architecture

Outline:

- Announcements
- Where do we go from here?
- Input/Output
- Devices
- Deadlock and its avoidance
 - Deadlock Avoidance Methods
- Banker's Algorithm (Dijkstra, 1965)
 - Single Resource

12.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- Old exams on the web site (warning...)
- Just a reminder that you should be reading T&W. Remember that chapter 2 reading is slow going.

12.2 Where do we go from here?

Three major operational areas for any operating system: (specializations of magic?)

IO How does information get in and out of the system and get where it's going.

Memory Management Making sure that things are where programs expect them to be.

Filesystem Making sure things stay where put and that everything operates efficiently.

12.3 Input/Output

Without IO, there's no real point in doing the computation. It's also complicated.

As always, it's all about abstraction: keep the dirty machine details hidden.

12.4 Devices

The Unix/Minix modes, devices are classified as:

- block devices (e.g. disks)
- character devices (e.g. ttys)

Not always clear: networks? printers? tapes? clocks? To model deadlocks, we need to consider abstract **resources** of two types:

preemptable resource one that can be taken away from the process owning it with no ill effects.
(e.g. the CPU, memory)

nonpreemptable resource one that can not be taken away from the process holding it without causing an error. (e.g. a printer once it's started to print.)

To use a non-preemptable resource:

1. request the resource
2. use the resource
3. release the resource.

12.5 Deadlock and its avoidance

Since we're talking about sharing, we have to worry about deadlock.

Occasionally a process must be given exclusive access to some resource, e.g.:

- a scanner,
- cdrom,
- tape drive,
- mouse,
- printer(lpd is a user process),
- terminal(usually unnoticed), etc.

Imagine two processes:

- copy tape to cdrom
- copy cdrom to tape

oops. (see Fig. 19 for a resource graph description)

A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.

Conditions required for deadlock: (Coffman, *et al.*, 1971)

1. **Mutual exclusion** processes don't share
2. **Hold and wait** processes can hold a resource and make other requests
3. **No pre-emption** processes can't be forced to surrender a resource
4. **Circular wait** the circle must exist.

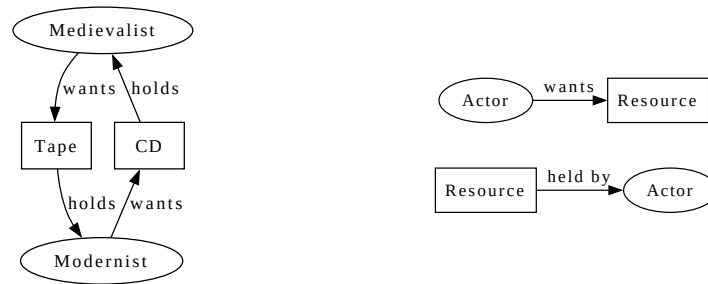


Figure 19: How a deadlock forms

12.5.1 Deadlock Avoidance Methods

- **Don't** the ostrich approach
- **Detection and recovery** discover deadlocks and kill jobs until it's gone. (carefully, or heuristically)
- **Prevention** place restrictions on accesses so that deadlock can never occur. (e.g. order resources)
- **Avoidance** Be very careful in resource allocation so it doesn't happen.

12.6 Banker's Algorithm (Dijkstra, 1965)

Keep track of the current state of resource allocation and only approve resources that could not make any one customer's requests unsatisfiable. (approve if there is a path that gets all customers to their credit limits.)

Each process must publish its maximum requirements up front.

12.6.1 Single Resource

Analogous to a line of credit: Bankers keep on hand much less than the sum of the maxima. The OS can, too.

A state is considered **safe** if there exists a sequence of states that leads to all customers getting their maximum loans. (resources)

The process:

1. Choose a process whose resource requirements can be satisfied. (It doesn't matter which, because it always increases the resource pool.)
2. Assume its resources are released (because it's finished)
3. Repeat until all processes terminate or there are no more satisfiable processes.

If all processes can terminate, the state **safe**. If not, it is **unsafe** and the resource request must be delayed.

Consider the following example in Figure 20 (from Tanenbaum).

Initial condition:

Name	Used	Max.	Needed
A	0	6	6
B	0	5	5
C	0	4	4
D	0	7	7

Available: 10

Safe: any order.

In progress:

Name	Used	Max.	Needed
A	1	6	5
B	1	5	4
C	2	4	2
D	4	7	3

Available: 2

Safe: $C \rightarrow D \rightarrow B \rightarrow A$

After B requests one more unit:

Name	Used	Max.	Needed
A	1	6	5
B	2	5	3
C	2	4	2
D	4	7	3

Available: 1

Unsafe: Nobody

Figure 20: One dimensional example of Dijkstra's Bankers Algorithm

Initial condition:

Name	Used	Max.	Needed
A	0	10	10
B	0	2	2
C	0	4	4
D	0	9	9
E	0	3	3

Safe: any order.

Available: 11

In progress:

Name	Used	Max.	Needed
A	2	10	8
B	1	2	1
C	2	4	2
D	2	9	7
E	2	3	1

Safe: $C(4) \rightarrow B(5) \rightarrow E(7) \rightarrow D(9) \rightarrow A(11)$

Available: 2

After A requests one more unit:

Name	Used	Max.	Needed
A	3	10	7
B	1	2	1
C	2	4	2
D	2	9	7
E	2	3	1

Unsafe: $B(2) \rightarrow C(4) \rightarrow E(6) \rightarrow \text{Nobody}$

Available: 1

Figure 21: Another example, but one that doesn't wedge immediately.

13 Lecture: Minix IO, cont.

Outline:

- Announcements
- Managing Multiple Resources
 - Resource Trajectories
- Managing Multiple Resources, cont.
 - Multi-way bankers'
- Wrapping up deadlock avoidance
- Back to I/O
- Devices
 - Low level considerations: timing, interleaving, etc.
 - Accessing a device: Device Controllers
 - Reading from a device: DMA vs. Programmed IO

13.1 Announcements

- Coming attractions:

Event	Subject	Due Date			Notes
asgn3	dine	Fri	May 1	23:59	
lab03	problem set	Mon	May 4	23:59	
midterm	stuff	Wed	May 6		
lab04	scavenger hunt II	Wed	May 13	23:59	
asgn4	/dev/secret	Fri	May 22	23:59	
lab05	problem set	Mon	Jun 1	23:59	
asgn5	minget and minls	Wed	Jun 3	23:59	
asgn6	Yes, really	Fri	Jun 5	23:59	

Use your own discretion with respect to timing/due dates.

- Old exams on the web site (warning...)
- Cleaning up after yourself.
 - `ps -u username`
 - `killall -u username -r pn-cs453/lib`
- Style guide:
 - Clean build
 - Magic numbers
 - Long lines (`~pnico/bin/longlines.pl`)
 - **not checking error returns**
 - A brace is the last thing on a line...

These will *always* cost points unnecessarily.

13.2 Managing Multiple Resources

But multiple types of resources are a problem...

13.2.1 Resource Trajectories

The example from Tanenbaum, p.249 is shown in Figure 22.

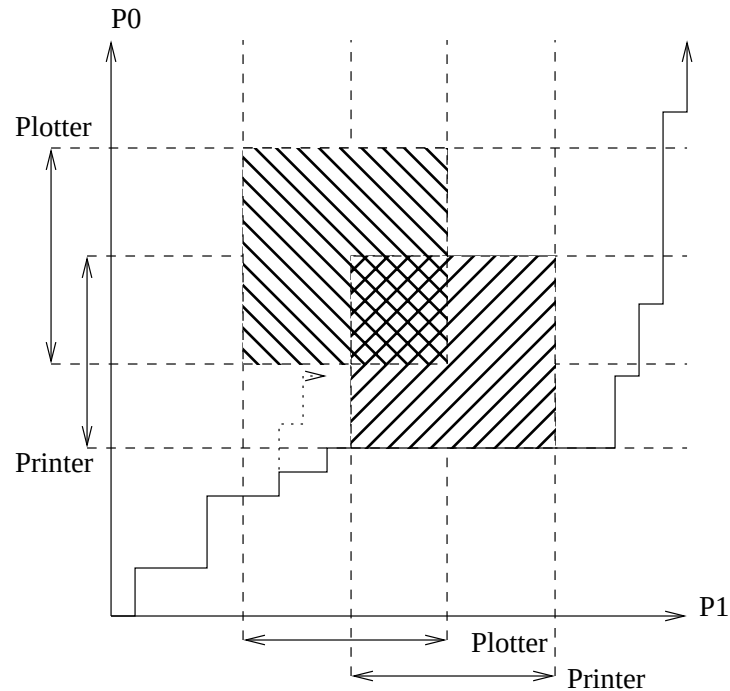


Figure 22: Resource trajectory example from Tanenbaum

Yuck.

13.3 Managing Multiple Resources, cont.

13.3.1 Multi-way bankers'

Here we simply do a vectorized version of the Banker's Algorithm:

- Maintain a table of held resources
- Maintain a table of maximum requests
- Maintain a table of remaining requests
- Maintain a vectors of allocated, free, total resources

If there does not exist a row less than the available vector, the system will deadlock, else:

1. Choose a process whose resource requirements can be satisfied. (It doesn't matter which, because it always increases the resource pool.)
2. Assume its resources are released (because it's finished)
3. Repeat until all processes terminate or there are no more satisfiable processes.

If all processes can terminate, the state **safe**. If not, it is **unsafe** and the resource request must be delayed.

See the example in Figure 23. Manipulations:

1. Initial situation
2. B requests an instance of R_2 (printer?): (safe: D, then A or E, then...)
3. E requests the last R_2 printer: **unsafe**

13.4 Wrapping up deadlock avoidance

Are any of these any good?

- Processes rarely know their resource requirements in advance.
- Processes come and go.

This is *hard*.

If you have a good idea, you can be famous like Dijkstra. :)

What does Unix do?

Not a thing. The Unix way (as with many other operating systems) is to hope it doesn't happen, and if it does, it expects some higher being (super user) to fix it. Think *Deus extra machina*.

13.5 Back to I/O

Without IO, there's no real point in doing the computation. It's also complicated.

As always, it's all about abstraction: keep the dirty machine details hidden.

13.6 Devices

The Unix/Minix modes, devices are classified as:

- block devices (e.g. disks)
- character devices (e.g. ttys)

Not always clear: networks? printers? tapes? clocks?

13.6.1 Low level considerations: timing, interleaving, etc.

Isn't it nice the controller can take care of it?

(under each level is another nice level of abstraction)

13.6.2 Accessing a device: Device Controllers

Mercifully the OS talks to device controllers not the actual devices. Isn't abstraction great?

Device controllers abstract away much of the complexity. Accessed via:

Memory-Mapped IO Device control registers are mapped into memory. (creates "holes" in memory)

	R_0	R_1	R_2	R_3
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Assigned Resources

	R_0	R_1	R_2	R_3
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources Still Needed

Exists: (6 3 4 2)
 Alloc.: (5 3 2 2)
 Free: (1 0 2 0)

(a) Initial Configuration

	R_0	R_1	R_2	R_3
A	3	0	1	1
B	0	1	1	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Assigned Resources

	R_0	R_1	R_2	R_3
A	1	1	0	0
B	0	1	0	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources Still Needed

Exists: (6 3 4 2)
 Alloc.: (5 3 3 2)
 Free: (1 0 1 0)

(b) After B requests an instance of R_2
 (Safe: $D \rightarrow A \rightarrow E \rightarrow \dots$)

	R_0	R_1	R_2	R_3
A	3	0	1	1
B	0	1	1	0
C	1	1	1	0
D	1	1	0	1
E	0	0	1	0

Assigned Resources

	R_0	R_1	R_2	R_3
A	1	1	0	0
B	0	1	0	2
C	3	1	0	0
D	0	0	1	0
E	2	1	0	0

Resources Still Needed

Exists: (6 3 4 2)
 Alloc.: (5 3 4 2)
 Free: (1 0 0 0)

(b) After E requests an instance of R_2
 (Unsafe: No process can be satisfied)

Figure 23: Multi-dimensional Bankers Algorithm

IO Ports Device control registers must be accessed through special instructions. (convenient, but complicates the CPU).

Either way, we set a value in some register, then the controller does the thing, then it sets a register to tell us that it's done it.

13.7 Reading from a device: DMA vs. Programmed IO

Standard (“Programmed IO”) Controller interrupts, CPU copies data from controller to memory.

DMA Controller copies data to memory, then interrupts.

At least we have interrupts (Think about the world if we didn't. We'd just have to check again and again, called “polling”).

14 Lecture: IO Software

Outline:

- Announcements
- Back to I/O
- Devices
 - Low level considerations: timing, interleaving, etc.
 - Accessing a device: Device Controllers
- Reading from a device: DMA vs. Programmed IO
- Goals of IO Software
- IO Software
- Minix IO Structure
- Minix IPC
- Structure of Minix: Client/Server model
 - Interrupt Handlers
 - Simple
 - complex
- Structure of Minix: Device Tasks (Drivers)
- Driver Behavior
 - Example: Disk seek optimization

14.1 Announcements

- Coming attractions:

Event	Subject	Due Date		Notes
asgn3	dine	Fri	May 1	23:59
lab03	problem set	Mon	May 4	23:59
midterm	stuff	Wed	May 6	
lab04	scavenger hunt II	Wed	May 13	23:59
asgn4	/dev/secret	Fri	May 22	23:59
lab05	problem set	Mon	Jun 1	23:59
asgn5	minget and minls	Wed	Jun 3	23:59
asgn6	Yes, really	Fri	Jun 5	23:59

Use your own discretion with respect to timing/due dates.

- -fcommon
- Precision of expression is important
- Midterm Wednesday
- Old exams to web (and warning)
- Lab03 due Mponday
 - I will release lab solns right after late days run. Perhaps before. Don't read them if not done.
- Exam Wednesday
 - Bring questions Monday; I'll leave time.

- For Asgn4, all of Minix’s driver behavior is explained in great detail in Chapter 3. Also `ipc.h` and `com.h`.

14.2 Back to I/O

Without IO, there’s no real point in doing the computation. It’s also complicated.

As always, it’s all about abstraction: keep the dirty machine details hidden.

14.3 Devices

The Unix/Minix modes, devices are classified as:

- block devices (e.g. disks)
- character devices (e.g. ttys)

Not always clear: networks? printers? tapes? clocks?

14.3.1 Low level considerations: timing, interleaving, etc.

Isn’t it nice the controller can take care of it?

(under each level is another nice level of abstraction)

14.3.2 Accessing a device: Device Controllers

Mercifully the OS talks to device controllers not the actual devices. Isn’t abstraction great?

Device controllers abstract away much of the complexity. Accessed via:

Memory-Mapped IO Device control registers are mapped into memory. (creates “holes” in memory)

IO Ports Device control registers must be accessed through special instructions. (convenient, but complicates the CPU).

Either way, we set a value in some register, then the controller does the thing, then it sets a register to tell us that it’s done it.

14.4 Reading from a device: DMA vs. Programmed IO

Standard (“Programmed IO”) Controller interrupts, CPU copies data from controller to memory.

DMA Controller copies data to memory, then interrupts.

At least we have interrupts (Think about the world if we didn’t. We’d just have to check again and again, called “polling”).

14.5 Goals of IO Software

abstraction	Abstract away the complexity to achieve device independence.
organization/standardization	It should be easy to name something. “/etc/motd”, rather than /dev/hda1/..., or, worse, (3,1)
error handling	Fix it ASAP, or report it in a useful way.
synchronization	Make operations make sense. (e.g. sharing a printer.)

14.6 IO Software

As always, it’s all about abstraction: (note the correspondence to the layers in Minix)

1. Interrupt handlers (bottom)
2. Device drivers — knows about the device
3. Device-independent OS software — does not know about the device
4. User-level software (top)

14.7 Minix IO Structure

We discussed principles of resource management, but how does this really work in the Minix environment?

The IO architecture of MINIX is shown in Figure 24

User Processes	make IO call; format IO; spooling
Device-independent software	Naming, protection, blocking, buffering, allocation
Device drivers	set up device registers; check status
Interrupt Handlers	wake up driver when IO completed
Hardware	perform IO

Figure 24: Minix IO

Corresponds to:

- user processes
- servers
- tasks
- process management

14.8 Minix IPC

Three primitives of Minix IPC:

Synchronous

```
send(dest, &message);
receive(source, &message);
send_rec(dest_src, &message);
```

Asynchronous

```
notify(dest);
```

Messages (other than `notify()`) communicated via **rendezvous** semantics: The sender waits until the receiver gets the message. Notify is asynchronous.

Why?

- simplifies buffer management.
- more predictable: there is no question of a program behaving differently given a different buffer size (out of its control)

(How would one find out the buffer size?)

14.9 Structure of Minix: Client/Server model

Unlike a monolithic system where, logically, each process has a user level existence and a projection into the kernel. In a Client/Server system, almost all of the parts of the operating system are user-level processes that communicate via message passing.

This provides:

- Modularity
- Robustness
- Security since the amount of trusted software is minimized.

14.9.1 Interrupt Handlers

- Disk
- Clock

Do a little work, then *tell someone*.

Can be simple or complex: (not always the way you'd think)

14.9.2 Simple

- disk: handler acknowledges interrupt and passes the word on to the device driver.
why? These calls are infrequent, long term, and involve a lot of work.

14.9.3 complex

- clock: counts ticks `pending_ticks` and only wakes up handler if necessary. (Clock frequency 60hz, quantum 100ms, 6/schedule interval . (See comment in `clock.c`)

wakes up driver if tty event or `SCHED_RATE`

why? These calls are frequent, high overhead, not much to be done.

```
* pending_ticks:
*   This is protected by explicit locks in clock.c. Don't
*   update realtime directly, since there are too many
*   references to it to guard conveniently.
* lost_ticks:
*   Clock ticks counted outside the clock task.
* sched_ticks, prev_ptr:
*   Updating these competes with similar code in do_clocktick().
*   No lock is necessary, because if bad things happen here
```

```

*      (like sched_ticks going negative), the code in do_clocktick()
*      will restore the variables to reasonable values, and an
*      occasional missed or extra sched() is harmless.
*
* Are these complications worth the trouble? Well, they make the system 15%
* faster on a 5MHz 8088, and make task debugging much easier since there are
* no task switches on an inactive system.

```

- tty: (keyboard) each key event causes an interrupt (down/up) keep track of events (bochs?)
More importantly: IO Processing state machine

14.10 Structure of Minix: Device Tasks (Drivers)

Device drivers are independent processes.

Drivers are all structured as:

```

init();                /* set up the device */
while(TRUE) {
    receive(any,message); /* receive a message from anyone */
    switch(mess.m_type) {
        ...            /* do the appropriate thing based on type */
    }
    send(mess.m_source,reply); /* say that you've done it */
}
shutdown();           /* clean up and go home */

```

14.11 Driver Behavior

Minix's device drivers are very simple so that we can get our heads around them. What would we do if we were working on a "real" system.

14.11.1 Example: Disk seek optimization

Device driver behavior can be very complex for optimization:

Consider the issues involved in disk access: seek time, rotational latency, transfer time....

Candidate algorithms:

- First come, first served
- Shortest seek first
- elevator algorithm
- Further: pick up "blocks of opportunity"; try to predict